claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_f10e69b6-333\agent-aa19895f4e2e52121.jsonl`
- SHA-256: `efa953d4835d4d49e3d1a13ec6bb8b8d5a8966d66cb584aa5f7f0ce6f7bb4874`
- Source modified: `2026-06-15T14:59:53+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_f10e69b6-333`
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`

Transcript

1. user (2026-06-15T14:57:50.100Z)

```
You are reviewing a DRAFT product-overview document for "KhelSutra" ? a local, AI badminton
highlight/rally tool.
The draft lives at C:/Users/avidu/Projects/badminton-highlight-
indexer/docs/KHELSUTRA_PRODUCT_OVERVIEW.md (Read it). It is dual-audience: (a) a non-technical
FIELD ASSOCIATE who will talk to
academies, and (b) invited ALPHA/BETA academies/coaches/players who will use the tool and give
feedback. It must be
PRODUCT-LEVEL (no fine-grained technical detail) and HONEST (no overpromising to real
prospective customers).

GROUND TRUTH (from the repo's docs/memory ? what is actually real):
- TODAY the tool genuinely does: upload a long video ? automatic rally detection ? highlight
reel (with a
  "Generated by KhelSutra.guru" watermark) ? a simple summary (rally count + total play time);
re-process/download;
  private local-first web app (async jobs + chunked upload + highlight download are live &
E2E-verified); runs on real
  4K GoPro match footage.
- Quality reality (DO NOT put raw numbers in a customer doc): clean single-court footage works
very well (~0.93 F1);
  busy MULTI-COURT footage is markedly harder (~0.66) and is the explicit moat/focus. The doc
should convey this
  qualitatively ("very good on clean single-court; multi-court is harder and is what we're
improving").
- ROADMAP (NOT today): shot detection (smash/drop/serve), deeper per-rally/per-match analysis,
per-player history,
  "AI assistant coach", more sports (table tennis next), Kannada/Hindi localization.
- THE FLYWHEEL (the headline the owner wants foregrounded): players/coaches can run their OWN
videos; coaches can
  correct what it found; over time + more videos the model sharpens AND that library unlocks the
deeper analysis.
- GUARDRAILS (hard, non-negotiable, must be represented accurately): using a user's video to
improve/train the model
  is OPT-IN (explicit permission), never automatic; footage involving MINORS is NEVER used for
model improvement;
  private/local-first ("your video, your data"); pricing is NOT set.
- Capture quality is the gate: fixed camera behind court, full court in frame, landscape,
steady, decent light;
  handheld/panning/broadcast footage is the failure mode.
- It is NOT line-calling/scoring/umpiring, NOT player biometrics, NOT a coach replacement.
RULES: read-only; you cannot edit. Return STRUCTURED output (consumed programmatically). Quote
the draft line/phrase
when flagging. Be specific and actionable. Do not invent product capabilities.
```

LENS: GROUNDING / FACTUAL ACCURACY vs the repo. Read the draft AND cross-check against the actual repo docs
(docs/NEXT_STEPS.md, docs/COMMERCIALIZATION.md, docs/FIELD_VISIT_PLAYBOOK.md, docs/PRODUCT_FIELD_ASSOCIATE_FLYER.md,
README.md, memory if readable). Flag any claim not supported by what's actually built or planned, any inconsistency
with the existing field docs (naming, framing, the "researching not selling" stance, the capture rules, the watermark
text), and confirm the flywheel/AI-coach framing matches the real owned-model roadmap (it does NOT auto-fine-tune
per-user today). Flag fine-grained technical detail that leaked in and should be removed for a product doc.

2. assistant (2026-06-15T14:57:53.472Z)

I'll start by reading the draft document and cross-checking against the repo's actual docs.

3. user (2026-06-15T14:57:54.453Z)

```
1       # KhelSutra ? Product Overview & Early-User Guide
2
3       *What it does today, how to get the best out of it, and where it's going.*
4       *For field associates and for invited alpha / beta academies, coaches, and players.*
5
6       ---
7
8       ## In one line
9
10      **KhelSutra turns ordinary badminton video into useful pieces ? automatically.** Point a phone or
11      GoPro at the court, record a session, and KhelSutra finds every rally, stitches a **highlight reel**,
12      and gives you a **simple summary** (how many rallies, how much real play time). No special hardware,
13      no editing, no manual scrubbing.
14
15      ---
16
17      ## The problem we're solving
18
19      Academies and serious players already record a lot of video ? and then almost never watch it. Hours
20      of footage pile up on phones and memory cards because cutting it into useful clips by hand is tedious
21      and nobody has the time. The good rallies, the moments worth reviewing, stay buried.
22
23      And most academy halls run **several courts at once**, so a single recording is a busy, noisy scene.
24      Ordinary video tools choke on that. **Pulling the real rallies out of that footage ? automatically ?
25      is exactly what KhelSutra is built to do.**
26
27      ---
28
29      ## Who it's for
30
31      - **Coaching academies & coaches** ? turn raw session footage into review-ready clips and reels for
32        teaching, without an editor.
33      - **Serious players** ? get clips of your own rallies to review, keep, and share.
34      - Today's focus is **badminton academies in Bengaluru** ? that's who we're building with first.
35
36      ---
37
```

What it does **today**

This is what's working right now, in the private web app invited users can try:

- **Upload a full session or match video** (from a phone on a tripod, or a GoPro). Long videos are fine.
- **Automatic rally detection** ? it finds where each rally starts and ends, on its own.
- **Highlight reel** ? the rallies stitched into one clip you can watch and share, with a small
  "Generated by KhelSutra.guru" mark.
- **A simple summary** ? how many rallies were found and how much actual play time there was.
- **Re-use & download** ? process once, come back to it, download the reel.

That's the core loop: **record ? upload ? get back a reel + summary.** No editing skill needed.

How to get the best out of it (capture matters)

KhelSutra works best on a **steady, full-court recording** ? the same way a coach would set a camera
to review play. A few simple habits make a big difference to the results:

- **Put the camera behind the court, slightly elevated**, with the **whole court in frame**.
- **Keep it still** ? a fixed tripod beats a handheld or panning shot every time.
- **Shoot landscape**, with reasonable lighting.
- **Record the whole session** and upload it as-is ? KhelSutra does the cutting.

Footage that fights these habits (shaky handheld, heavy zooming/panning, broadcast-style cuts, very
dark halls) is the **hardest case** and where results suffer most. When in doubt: fixed camera, full
court, don't touch it.

> A one-page capture checklist is provided to academies in the trial.

The part that makes KhelSutra different: **it learns from your videos**

This is the heart of the product, and what sets it apart from a generic "AI clipper":

Your videos make _your_ tool sharper. As you and your players run more of your own footage through
KhelSutra ? and as coaches confirm or correct what it found ? the underlying model gets better at
your courts, *your* lighting, *your* players, and the busy multi-court halls that trip up everything
else. It improves with use instead of staying frozen.

That same growing library of real, labelled footage is also what unlocks the **deeper features below** ?
which is why early users aren't just customers, they're co-builders. Every session you contribute makes
the next version smarter.

Your data stays yours. Using your video to improve the model is **opt-in** ? nothing is used to train
anything without your explicit permission. We don't publish anyone's footage. **Sessions involving minors
are never used for model improvement.** The product is built **private and local-first**: your video, your

```
data, your control.

---

## Where it's going ? toward an **AI assistant coach**

Today KhelSutra finds rallies and makes reels. The roadmap ? fed by the growing video library above ?
turns it into a genuine training companion:

- **Shot detection** ? recognising smashes, drops, net shots, serves, and clears within each rally.
- **Deeper per-rally and per-match analysis** ? patterns, tendencies, what's working and what isn't.
- **Per-player & per-match history** ? track a player's progress over weeks and months, not just one clip.
- **Personalised insights** ? an **AI assistant coach** that supports (never replaces) the real coach,
  surfacing what to work on next.
- **More sports** ? table tennis next, then other racket/court sports.
- **Your language** ? Kannada and Hindi, alongside English.

> **Read this honestly:** the items in this section are the **direction**, not today's buttons. What you
> can use *today* is rally detection + highlight reels + the play-time summary. The deeper analysis arrives
> as the video library grows ? which is exactly why your footage and feedback matter so much.

---

## What we're focused on over the next ~4 weeks

1. **Growing the library of real academy footage** ? especially busy **multi-court** halls (the hard,
   valuable case). This is the single biggest lever on quality.
2. **Field conversations with academies** ? understanding how coaches and players actually work with
   video, and what would make this genuinely useful.
3. **Recording sessions and showing academies their own footage** as reels ? *"this is what it looks
   like for you."*
4. **Sharpening multi-court accuracy** and inviting a few more academies into the private trial.

---

## What KhelSutra is **not** (so expectations stay honest)

- It's an **early, invite-only trial** ? useful, but still improving; you'll hit rough edges.
- It is **not a line-calling, scoring, or umpiring system** ? it finds rallies and highlights.
- **Shot detection and the deeper "AI coach" analytics are on the roadmap, not available today.**
- It is **not a replacement for a coach** ? it's a tool that saves coaches time and adds insight.
- Accuracy depends on **capture quality**; messy handheld/broadcast footage is a known weak spot.
- **Pricing isn't set** ? we're still learning what's fair; today's invited use is about getting it right.

---

## Quick answers to common questions
```

```
134
135     **"How accurate is it?"**
136     Very good on a clean, single-court recording. A busy multi-court hall is harder ? and is
exactly what
137     we're improving. The best answer is to *show* a sample reel rather than quote a number.
138
139     **"Do I need a special camera or hardware?"**
140     No. A phone or GoPro on a tripod, behind the court, whole court in frame. We provide a
one-page checklist.
141
142     **"Is my video private? What happens to it?"**
143     It's built private and local-first ? your video stays yours. We never use anyone's
footage to improve the
144     model without explicit opt-in, we don't publish footage, and **footage involving minors
is never used**.
145
146     **"Can it tell me the score / who won / call lines?"**
147     Not today. Today it finds rallies and builds a highlight reel + a play-time summary.
148
149     **"Does it work for doubles and tournaments?"**
150     Yes for finding rallies. Crowded multi-court tournament halls are the hard case we're
specifically working on.
151
152     **"Will it get better?"**
153     Yes ? that's the whole idea. The more real footage runs through it (with permission) and
the more coaches
154     correct it, the sharper it gets, and the sooner shot detection and deeper analysis
arrive.
155
156     **"Which sports?"**
157     Badminton today; table tennis and other racket sports are on the roadmap.
158
159     **"When can I use it / how do I join?"**
160     It's an invite-only private trial. If you're interested, the best next step is to let us
record a session
161     and show you your own court and players as a reel.
162
163     ---
164
165     *KhelSutra · khelsutra.guru · Internal & invited-trial overview, June 2026.*
166     *Field associates: the conversation script and visit rules live in the separate Field
Visit Playbook.*
167
```

4. user (2026-06-15T14:57:54.942Z)

```
1       # Next steps (live) ? owned-model + multisport
2
3       **Updated:** 2026-06-13 (**rally-detection quality track**: fusion + experiment-harness
design docs merged,
4       owner-gated ? see the dedicated section below, the desktop-agent pick-up). Prior:
2026-06-10 (audit?M0?Gemini-battery session; later same day the **UI-alpha track opened**:
5       ADR-009 ratified ? KhelSutra Guru / khelsutra.guru, private `khelsutra` repo, local-
first delivery ? see
6       `docs/UI_ALPHA_EXECUTION_PLAN.md`; **PR-1 async jobs (#16) MERGED 2026-06-11, PR #87**;
**PR-2
7       upload/download MERGED 2026-06-11, PR #99** ? live-E2E-verified (3-part chunked upload
MD5 byte-identity +
8       compile?browser-download); next: PR-3 identity. Same day: the **8?9 quality sweep
#90?#98 landed** ? ruff/mypy/
9       coverage-floor CI lanes, SQLite conn-close fix, config unknown-key warning, hybrid-twin
collapse into
10      `trajectory_hybrid.py`, WSL tilde+abspath live-path fixes; suite 388 green; LOVO 0.626
re-verified exact).
11      **Authoritative roadmap:**
```

`docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` (+ ADR-008 for topology). **Latest blow-by-blow:** memory
`current-state.md` / `session-handoff.md`. This file = the prioritized "what to do next," refreshed each session.

? Next-session / GPU-box pickup (2026-06-15)
M0 in-container quick wins are COMPLETE: ship-gate provisional blocker (**#170**), train/eval split guard (**#171**), the **doc-consistency sweep** (**#174** ? corrected 16 drifted docs vs the code), and the **P3 person-feature fusion litmus machinery is already present + suite-green** (`backend/eval/fusion_compare.py` + `ablation.py`: person-driven ?F1 / bootstrap CI / paired sign test; the real-golden `skipif` litmus waits only on GPU-extracted data). ? CI is red repo-wide due to a **GitHub Actions minutes/billing outage** (account-level, not code) ? work is local-gated + `--admin`-merged until Actions is restored (Settings ? Billing ? Actions minutes / spending limit).

Immediate next step ? the real P3 LOVO measurement, split GPU?CPU (the dev box is CPU-only: `torch+cpu`/`cv2` present but no GPU, and the corpus/videos/CSVs/labels are not here):
1. **[owner / GPU box]** `python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json --out-dir "%USERPROFILE%\OneDrive\rally-annotator\golden-data\features\2026-06-15-n6"` ? produces `*_golden_features.json` for the 6 golden videos. (GPU + WSL + videos + trajectory CSVs + `.rallies.csv` labels are owner-side only.)
2. Artifacts land in the **shared cloud folder** `C:\Users\avidu\OneDrive\rally-annotator\golden-data\` (OneDrive auto-syncs both ways). Convention + workflow: **[`GOLDEN_DATA_SHARING.md`](GOLDEN_DATA_SHARING.md)**; tracked in **#175**.
3. **[CPU dev/agent session]** `python -m backend.eval.fusion_compare --features-dir <shared>\features\2026-06-15-n6 --record` + `python -m backend.eval.ablation --features-dir <shared>\... --granularity group` ? real person-feature ?F1; copy the JSONs into `output/` to also pass the `skipif` litmus tests (`tests/test_ablation.py::test_litmus_reproduces_gate_a`).

Also queued (owner-gated / pending input):
- Ship-gate target calibration ? **#172** (owner decision; needs corpus growth).
- Tests reorganization ? blocked on the owner's grouping metric (provide it ? an agent will do it).
- Restore CI ? the Actions outage is account-level (billing/minutes), not a repo/config bug.

Leaving (bulky / M2 ? no start without owner go): ActionFormer (M0.4), event-index DB migration (M0.1), Gemini-silver purge (M0.3), cost-to-Gen-2 benchmark + ByteTrack deprecation, multisport, repo rename, PMF execution.

Where we are (one paragraph)
ADR-008 ratified ("repo split driven by productionization, not isolation"): training lives in-repo (`training/`)
behind a CI-enforced import wall; the featurizer is single-sourced (`backend/features/rally_seq.py`, train==serve);
the **Gen-0 harness ships** (`python -m training.gen0.harness --manifest output/human_lovo_manifest.json --floor
eval_baselines/heuristic_lovo_n6.json --version <semver>`) and produced artifact v0.1.0 (LOVO **0.626**, floor passed,
sign test 5/6 wins p=0.109 ? honestly not significant, ship=False). The **n=6 owned corpus is in the collector**
(262 rallies, Proprietary, commercial export = owned data only after the ShuttleSet reclassification). The **Gemini
battery is done** (see table) ? the moat is quantified.

The quality table that now drives strategy (IoU 0.5 vs human golden labels)
Path	Mahadevpura (clean-ish)	Kushagra (multi-court)
Heuristic (velocity windowing)	0.657	0.157
Gen-0 owned head (LOVO, n=6)	0.744	0.561
Two-stage WASB+Gemini refine	0.83	?
Gemini wrapper (`gemini-flash-latest`)	**0.93**	**0.66**

Reading: clean footage is commoditized (serve it with Gemini for cents). **Multi-court drops the commodity

wrapper to 0.66 ? that is the owned model's ship target** (its FPs are background-game pickups + dead-time merges,
the exact contrast-metric confound). Gen-0 is 0.10 behind with only n=6 ? a corpus-growth-sized gap, not an
architecture-sized one. Two-stage refine = the cost-efficient serving path for LONG tapes (uploads candidate clips
only); wrapper 503-flakiness (observed all session) makes the provider-fallback registry load-bearing.
Baselines tracked in `eval_baselines/` (heuristic_lovo_n6.json, gemini_wrapper_2026-06-10.json).

Rally-detection quality track (design merged 2026-06-13, owner-gated; implementation in progress)

This is the track a desktop/local agent is slated to pick up. Design docs landed (PR #112 + session). All implementation is **owner-gated** (per explicit in the plan); each step = ONE PR off `master`.

Hardening loop (T1-T5 positive tests + audit + finale) COMPLETE (2026-06-14, PRs #156/#157/#160/#161/#163 + #165 + final records). See archived `docs/archives/CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md` (Living Log with every pre/post/review) and `docs/archives/HARDENING_LOOP_HANDOFF-COMPLETED-2026-06-14.md` (full rules + STATE). Top-level now pointer stubs per #165 review. All 5 items + verification + archive done; clean slate.

- **Docs:** `docs/MULTI_SIGNAL_FUSION_PLAN.md` (shuttle + person + audio, one fusion layer) ·
  `docs/EXPERIMENT_HARNESS.md` (A/B + shadow + promotion gate, zero-regression) ·
  `docs/ROORA_LABELING_GUIDELINES.md` (v8) · `docs/HOW_RALLY_DETECTION_WORKS.md` (2-layer mental model).
- **Key finding (no re-investigation needed):** the owner's "held-shuttle counted as a rally" bug is **NOT a missing capability** ? `backend/pipeline/detectors/rally_gate.py` already implements the net-crossing **"Gate A"** (`apply_gate(..., min_crossings=2)`) that kills service-feed/held-shuttle windows, but it is **only called from eval drivers** (and there was no `min_crossings` knob in `IndexingConfig`). **Wiring Gate A into the live segmenter (now via fusion_hybrid) was the single cheapest, highest-confidence quality win.** (P2 FusionSegmenter specifics: registry `fusion_hybrid`, **default-OFF**, seeds from substrate, persists `net_crossings` + optional person features, optional served Gate A/B via config knobs; adversarially reviewed; suite green. See `tests/test_served_gate_a_regression.py` for the honest finding that on production windowing Gate A is neutral/negative in some cases, hence kept opt-in with knob for safety, and the leaderboard revert note.)
- **Current status (2026-06-14):** P1 (person cue extraction to reusable `PersonDetector`) + P2 (FusionSegmenter + Gate A signal + served handoff gating) + experiment harness P1 DONE (adversarial review + NO-REGRESSION; suite green). **P3 next (this PR + follow-ups):** learned fusion (person features ? harness `compare` LOVO lift on the 6 golden videos; eager-person-compute optimisation); then P4 audio via hit_detector.
- **Pick-up order (each = ONE PR off `master`, owner approval first, flag-gated default-OFF, promote only on measured LOVO):**
  1. (P2 first-step, largely done via fusion_hybrid) Wire Gate A into production served path + config knob (min_crossings); apply in hybrids at runtime. (Now available opt-in; served neutral on production windowing per litmus ? kept opt-in with knob for safety.)
  2. P3 first step (this work item): harness `compare` LOVO litmus for person-on variant (players_in_court + two_sided + colocation etc. from `fusion_features`) on the 6 golden; add the eager compute optimisation note/landing. Pure + harness (testable here).
  3. P3/P4 continuation: audio cue integration + full learned promotion if lift proven.
- **Discipline (from EXPERIMENT_HARNESS.md):** every new cue **flag-gated, default OFF**; promote to default **only on measured LOVO lift** through `run_regression.py` (exit 0/1/3); pinned `CONTROL` ? rollback = config flip. Harness re-uses ~80% existing code.
- ? Dev container no GPU/torch/cv2 ? Gate A, person cue logic, harness, and pure fusion paths are testable here; real-video / golden LOVO / ablation confirmation is owner-side (GPU/WSL machine).

Discipline reminder: owner approval before starting any owner-gated item; one PR per step; babysit the PR (poll/fix/reply) until merge observed, *then* advance.

Prioritized next steps

1. **[owner, in progress] GROW THE GOLDEN CORPUS** ? the single highest-leverage move; every lever below feeds on it.
   Priority order for new videos: **(a) multi-court badminton** (the target distribution AND where the ship target
   lives), **(b) ?3 matches per new sport ? table tennis first** (WASB transfers at F1 0.909; pickleball labels are
   corpus-gold but not trainable until a clean MIT/Apache ball detector exists), (c) capture variety per
   `VIDEO_RECORDING_GUIDELINES.md`; **adult sessions only for the training pool** (consent guardrail). Per video:
   label ? `curate import-local --normalize --license Proprietary --fps <true fps>` ? re-run the Gen-0 harness.
   The paired sign test needs n: at n=10 with 9 wins, p?0.011 ? significance is reachable this month.
2. **[owner decision] Set the ship-gate target** ? now informed: floor = heuristic 0.480 (necessary), **multi-court
   reference = wrapper 0.66** (the number that makes the owned model worth serving), clean-footage ceiling = 0.93
   (not the target ? Gemini serves that tier). Suggested shape: "LOVO F1@tIoU0.5 ? 0.70 on multi-court folds with
   paired-sign p<0.05 vs heuristic" ? owner to ratify/adjust.
3. **M0.4 next increment ($0/local):** swap the Gen-0 LogReg for **ActionFormer/ASFormer** (MIT, minutes on the
   2070S ? still lightweight, NOT the heavyweight Gen-2 stack) inside the existing harness; per-video threshold
   calibration (the LOVO-vs-oracle gap is visible per fold in the harness output).
4. **Multisport thread:** ingest Roora pickleball/TT CSVs (`import-local --normalize`); merged-prototype LOVO across
   badminton+TT once TT has ?3 matches; identify a clean pickleball ball detector. ? TT-on-WASB serving stays gated
   by C3 (provenance multiplies per sport).
5. **PMF validation (cheaper than any engineering month):** C13 academy interviews (+ the two added questions:
   "multi-court footage nobody watches?" / "pay per-match to index it?"); camera pilot at the warmest 2-3 academies
   (consent at capture; demo served via the Gemini path or human-polish ? the reel pattern exists:
   `output/MahadevpuraSingles_highlights.mp4`). **Field kit COMPLETE (2026-06-12, owner ratified the
   physical-visit approach; reframed product-feedback-first per owner spec):**
   `docs/PRODUCT_FIELD_ASSOCIATE_FLYER.md` (the recruiting flyer ? owner fills ?/contact placeholders and recruits) +
   `docs/FIELD_VISIT_PLAYBOOK.md` (the hired associate's visit playbook: coach + student script, demo-after-Q6
   rule, ground rules, Phase-2 record-and-show-back stretch goal) + `docs/FIELD_VISIT_ANSWER_SHEET.md` (per-visit
   capture form incl. demo-reaction + student blocks) + `docs/PMF_FIELD_SIGNALS.md` (G/A/R anchors + **pre-registered
   decision rules** ? PROCEED / PIVOT A-C / FALSIFIED thresholds fixed before the first visit).
6. **Housekeeping:** ~~rotate the Gemini API key~~ **DONE 2026-06-10** (owner-verified working; confirm old-key
   deletion in AI Studio); **rename repo off "badminton"** (owner action,
   in BACKLOG); collector=SoR confirmation; off-peak niceties: none pending ? the battery completed.

Pointers / artifacts

- Roadmap: `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` · Topology: ADR-008 (`docs/archives/decisions/DECISIONS.md`) ·
  Study: `docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md` · Quality history:
  `docs/archives/quality-iterations/2026-06-multivideo-lovo/`.
- Harness: `training/gen0/harness.py` (one command ? train ? LOVO ? gate ? artifact bundle; weights gitignored,
  manifest reviewable). Featurizer: `backend/features/rally_seq.py` (FEATURE_SCHEMA v1).
- Corpus: collector DB (system-of-record, 6 matches/262 rallies) · eval manifest

`output/human_lovo_manifest.json`
112   (gitignored) · durable trajectories + Roora files: `C:\Users\avidu\Projects\Annotation Setup\Collect\Trajectories\` ·
113   labels: `...\Collect\Golden Labelled\`.
114   - Gemini ops learnings: serial uploads (`indexing.ai_max_workers=1`) + chunk re-encode
115   (`indexing.ai_chunk_scale_width=1280`) + 6-attempt exponential generate retry ? a cold/prepaid project burst-throttles
116   with a MISLEADING "credits depleted" 429; ramp gradually.
117   - ? No owned-model implementation beyond greenlit M0.x until owner greenlights the next milestone. Committed next
118   PLATFORM slice = async job model (single-tenant).
119

5. user (2026-06-15T14:57:55.415Z)

1   # Commercialization Tracker
2
3   **Status:** ? LIVING ? track this in parallel with product/quality work.
4   **Owner:** avidullu · **Last refreshed:** 2026-06-07
5
6   **Purpose.** Single source of truth for the **commercial** dimension of the project:
7   licensing/legal posture, codec & output economics, hosting unit-economics, the
8   golden-set vendoring spend, packaging/distribution, and the product-market-fit
9   risks that gate monetization. Updated as decisions land ? see the
10   [Changelog](#changelog) at the bottom.
11
12   **Merged from / supersedes:**
13   - `archives/MONETIZATION_AUDIT.md` (2026-05-30 deep-research licensing & economics
14   audit) ? kept as the **research-of-record** (per-claim verification transcript,
15   primary sources). This doc carries its *current* status forward.
16   - `archives/COMMERCIAL_READINESS_REVIEW.md` (code-health + PMF assessment) ?
17   archived; its commercial findings + the quality-phase priorities live here now.
18
19   **Premise (unchanged, verified):** the product is a **closed-source hosted API** ?
20   users upload videos to a cloud NVIDIA GPU VM and get highlight reels back. We never
21   distribute code/binaries, so GPL/LGPL *distribution* copyleft is not triggered ? but
22   **AGPL §13 (network clause)**, **metered-API terms**, and **video-codec patent
23   royalties** all still apply.
24
25   ---
26
27   ## 1. Commercialization dashboard
28
29   Legend: ? done · ? in progress / watch · ? blocker / needs action · ?? needs legal counsel · ? future
30   Priority: **P0** (do first) · **P1** · **P2** (lowest). Rows are grouped **Pending ? In-flight ? Completed**, each sorted by priority. (C# IDs are stable identifiers ? they don't imply order.)
31
32 | # | Workstream | Pri | Status | Where it stands | Next action / owner decision |
33 |---|---|---|---|---|---|
34 | | **? PENDING (needs action / not started)** | | | | |
35 | C5 | **Output codec royalties** | **P0** | ? | Highlights are still encoded **H.264 + AAC** (`libx264` in `backend/pipeline/stitcher/compiler.py`) ? encode-side AVC patent exposure remains. | Switch output to **AV1 + Opus** (royalty-free) on an **Ada-class GPU (L4/L40)** for HW encode. See §4. Decision: provision Ada-class GPU? |
36 | C9 | **Packaging / distribution** | **P0** | ? | `pyproject.toml` (PEP 621, hatchling) added; the diagnostic `setup.py` renamed to `scripts/doctor.py`; `indexer` console entry point; CI installs `pip install -e .[dev]`. torch/torchvision stay an out-of-band `--index-url` install (PEP 621 can't pin CUDA wheels). | Implemented per `DEFERRED_HARDENING_PLAN.md` (#7). Remaining: publish `obsreport` so the `reporting` extra is pip-installable. |
37 | C12 | **SaaS foundations (async/auth/tenancy)** | **P1** | ? | Async job model shipped (`DEFERRED_HARDENING_PLAN.md` #16, PR #87); auth/tenancy still future. | Multi-user/platform strategy in `PLATFORM_ARCHITECTURE.md` (pluggable storage/compute, tenancy, orchestration). |
38 | | **? IN-FLIGHT (in progress / watch)** | | | | |

39 | C4 | **Runtime AI strategy (A/B/C + owned model)** | **P0** | ? | Live = **(C) Gemini 2.5 Flash, paid tier**. Long-term = **own a self-hosted generative video-QA model** (fine-tuned open VLM) as primary, **paid Gemini always-ready as fallback** via the provider registry. | Make "provider registry + paid fallback" a core principle; build the owned model **after** the scaffolding/flywheel are solid. See `PLATFORM_ARCHITECTURE.md` §5A. |
40 | C13 | **Beachhead validation** | **P0** | ? | Market research (`PMF_MARKET_RESEARCH.md`) recommends **India badminton academies** as the #1 beachhead. Unvalidated. | Cheapest tests: 10?15 academy/coach discovery interviews; rally-detection P/R benchmark on 100+ real amateur clips; fake-door pricing (B2C ~$99/yr, B2B academy ~$600/yr). See §6. |
41 | C8 | **Golden-set vendoring spend** | **P1** | ? | ADR-006: **Roora (Bengaluru)** selected for an initial paid pilot; licensed/owned video only (no user uploads to vendors). | Confirm quality-per-dollar from the pilot; then GS-3 `HumanVendorProvider`. See `GOLDEN_SET_VENDORING.md`. |
42 | C3 | **Shuttle-tracking weights provenance** | **P1** | ? ?? | WASB-SBDT **code is MIT (clear)**; distributed **checkpoints are academic-data-trained** ? not covered by the MIT code grant. MonoTrack/YOLO-NAS banned (noncommercial). | **Ship-gating** blocker (multiplies per sport). Resolve via an own-trained detector **or** a clean MIT/Apache detector swap ? NOTE: ADR-002 says our temporal labels can't train the detector (needs coordinate labels), so own-weights is a new, un-scoped workstream, NOT delivered by ROADMAP Phase 4. |
43 | C14 | **Owned-model base licensing + training-data policy** | **P1** | ? ?? | Owned video-QA model must build on a **commercial-clean (MIT/Apache-2.0/BSD), video-capable** base ? primary **Qwen3-VL (Apache-2.0)**, fallback **InternVL3 (MIT)**. **Gemma 4 = AMBER / bench-only** (Apache grant likely but Prohibited-Use posture unconfirmed ? counsel needed; not a build target yet). **Reject Gemma 3** (viral derivative clause ? permanent Google PUP), Llama Vision (naming + 700M-MAU), `gpt-oss` (text-only). **Hard rule: never train on paid-LLM (Gemini/OpenAI/Anthropic) outputs** (competing-model clause). The moat is the **consented dataset + eval harness**, not the model. | **MIT/Apache-2.0/BSD** for code+data+weights; **open-model teachers only**; separate explicit **training opt-in**; **exclude minors**; per-clip consent ledger. See `OWNED_MODEL_IMPLEMENTATION_PLAN.md` (authoritative). |
44 | C6 | **Inbound HEVC decode** | **P2** | ? ?? | Decoding users' GoPro H.264/**HEVC** uploads is unavoidable; HEVC pools are fragmented + highest-uncertainty. | Low/uncertain residual; confirm decode-side exposure with counsel before launch. |
45 | C7 | **Hosting unit-economics** | **P2** | ? | Per-video cost is *cheap* for Gemini (~$0.05 input for a 10-min clip); risk is volume × latency, not unit price. Self-host/offline costs unmeasured. | **Benchmark $/video & latency on the target VM** for A/B/C before pricing. See §5. |
46 | C10 | **Value layer (product surface)** | **P2** | ? | Minimal exports shipped (EDL/JSON/CSV + `basic_stats`, PR #51). | Deepen: per-rally tags/notes that feed golden, per-player/match stats, clip gallery. See §6. |
47 | C11 | **Input-quality enforcement** | **P2** | ? | Per-video observability reports surface a customer verdict + footgun flags incl. audio-present (PR #50). | Turn more `VIDEO_RECORDING_GUIDELINES` dimensions into validators/report rules. |
48 | C2 | **`supervision` ByteTrack pin** | **P2** | ? | Pinned `<0.30.0` (deprecation ahead). | Migrate before lifting the pin (BACKLOG). Not a legal risk, a maintenance one. |
49 | | **? COMPLETED** | | | | |
50 | C1 | **AGPL / copyleft removal** | ? | ? | ADR-005 shipped: AGPL `ultralytics` YOLOv8 replaced with **torchvision Faster R-CNN (BSD) + `supervision` ByteTrack (MIT)**. No AGPL code/weights in the stack. | Done. Keep new deps permissive. |
51
52   ---
53
54   ## 2. Licensing & dependency posture (refreshed risk table)
55
56   Verdicts updated against the current stack (`requirements.txt` + code). Original
57   2026-05-30 verdicts and the 25-claim verification transcript are in
58   `archives/MONETIZATION_AUDIT.md`.
59
60 | Dependency | License | SaaS obligation? | Current verdict | Notes |
61 |---|---|---|---|---|
62 | ~~`ultralytics` + `yolov8n.pt`~~ | AGPL-3.0 | Yes (§13) | ? **REMOVED** | Replaced per ADR-005 ? no longer in the stack. |
63 | **torchvision** detectors (Faster R-CNN/RetinaNet) | BSD-3 | No | ? LIVE | The chosen replacement; person = COCO class (see `backend/pipeline/detectors/person_detector.py` `_PERSON_CLASS_ID`). |

```
64      | **`supervision`** (ByteTrack) | MIT | No | ? LIVE | Pinned `<0.30.0` ? migrate before
bump (C2). |
65      | **WASB-SBDT** (shuttle) | MIT (code) | No | ? code clear | Weights provenance open
(C3). |
66      | **MonoTrack / YOLO-NAS** | noncommercial | ? | ? BANNED | Do not use. |
67      | **Gemini API** (paid tier) | service terms | Paid: no training/review | ? LIVE WATCH |
Must bill the paid tier to keep user video private (C4). |
68      | H.264 / HEVC / AAC codecs | patent pools | royalties on encode/decode | ? ?? |
Separate from ffmpeg's license. Encode-side open (C5); decode-side residual (C6). |
69      | ffmpeg/ffprobe, google-genai, torch, opencv, numpy,
fastapi/uvicorn/pydantic/jinja2/dotenv | LGPL/Apache/BSD/MIT | No | ? CLEAR | Invoked as
subprocess / imported; all permissive. |
70      | **`obsreport`** (reports) | permissive (private) | No | ? soft dep | Pin the
`git+ssh@vX` tag once the repo is published (BACKLOG). |
71
72      **Bottom line:** the two hard blockers from the audit (Ultralytics; banned weights)
73      are resolved/avoided. Remaining commercial-legal items are **codec royalties (C5/C6)**
74      and **weights provenance (C3)**, plus the metered-API privacy posture (C4).
75
76      ---
77
78      ## 3. Runtime AI strategy ? A/B/C
79
80      The semantic rally-boundary step is the one external/metered dependency. Three paths
81      (from the audit), with current standing:
82
83      - **(A) Eliminate ? offline learned segmenter.** ActionFormer (MIT) / our own learned
84        classifier on `candidate_segments.stats`. **Best steady-state margin** (no per-call
85        cost, no external dep) and matches the Phase-4 roadmap (substrate = WASB
86        shuttle-motion, decided in ADR-004). **Most engineering.** ? *target*
87      - **(B) Self-host VLM.** Qwen2.5-VL-7B/32B (Apache-2.0) or InternVL2.5-8B (MIT). No
88        per-call fees; pays in GPU VRAM (~16?24 GB for 7B). ?? Qwen 3B is noncommercial.
89      - **(C) Keep Gemini 2.5 Flash (paid).** Fastest, cheap per video, video-native;
90        ongoing metered cost + external dependency. **? current live path.**
91
92      Hosted alternatives for (C) comparison: Anthropic Claude (no training on
93      inputs/outputs; customer owns outputs; **no native video** ? weaker here);
94      OpenAI (no training by default; ZDR available); Gemini remains strongest video-native.
95
96      ---
97
98      ## 4. Codec / output economics (the live gap ? C5)
99
100     **The trap:** ffmpeg being free (LGPL) does **not** grant the *patent* rights to the
101     codecs it implements. Royalties attach to the act of encode/decode.
102
103     - **Encode side (we control it):** highlights currently emit **H.264/AAC**. AVC (Via LA)
104       is first 100k units/yr royalty-free with a permanent "free to end-users over the
105       internet" provision ? a small SaaS likely falls under thresholds, but ?? confirm.
106     - **Mitigation:** emit **AV1 (SVT-AV1, BSD + AOMedia royalty-free patent grant) + Opus
107       (royalty-free)**. NVENC AV1 HW-encode needs **Ada-class GPUs (L4/L40)**; Ampere/Turing
108       must use **SVT-AV1 (CPU)**.
109     - **Decode side (unavoidable, C6):** inbound GoPro **HEVC** decode ? fragmented pools,
110       highest uncertainty; low residual.
111
112     **Recommendation:** provision an **Ada-class GPU (L4/L40)** and standardize highlight
113     output on **AV1 + Opus** ? removes the encode-side royalty question almost entirely.
114     *Implementation lives in `backend/pipeline/stitcher/compiler.py` (codec args).*
115
116     ---
117
118     ## 5. Hosting unit-economics (C7)
119
120     Reference workload: ~30-min 1080p match on a cloud NVIDIA GPU VM.
121
```

- **(C) Gemini 2.5 Flash (paid):** input $0.30/1M tokens, output $2.50/1M; video ~263
  tok/s ? 10-min clip ? 158k input tokens ? **~$0.05 input**. Cheap per unit; the
  economic risk is **volume × latency**. Flash-Lite is cheaper; Pro ~10×.
- **(B) Self-host Qwen2.5-VL-7B:** $0 API; cost = GPU-hours / throughput; needs ?24 GB
  VRAM (L4/A10). $/video vs Gemini is **open ? measure**.
- **(A) Offline ActionFormer/learned:** cheapest at inference; cost = one-time training
  + light GPU. **Best steady-state margin.**

> The A/B/C $/video comparison depends on GPU instance + batch throughput ?
> **benchmark on the target VM before pricing the product.**

---

6. Product-market fit & the quality/data phase

Beachhead (from market research, 2026-06-07 ? see
[`PMF_MARKET_RESEARCH.md`](PMF_MARKET_RESEARCH.md))
The static-camera constraint is a **B2B filter**: the buyer must already record from a
fixed angle, which points away from the median casual player and toward coaches/clubs.
Ranked beachheads:
1. **Badminton coaching academies & serious players, India** *(strongest)* ? 20M+
players,
   formalizing academy ecosystem (Khelo India), genuine fixed-angle recording, coaches
   who already monetize, badminton whitespace vs incumbents. Sell to the **academy**,
not
   the individual; needs INR-tuned pricing **and a Kannada/Hindi product** ? see
   [`I18N_PLAN.md`](I18N_PLAN.md) (Bangalore-first localization, by design).
2. **US/global pickleball players & clubs** ? fastest-growing sport (19.8M US, +45.8%
YoY)
   but **SwingVision already owns single-camera tennis/pickleball AI** (USD 179.99/yr) ?
   head-to-head, not greenfield.
3. **Badminton academies/clubs in East/SE Asia + Denmark** ? highest density; China
needs
   local hosting/partners; Denmark/EU small but high-WTP, easy to pilot.

Pricing anchors *(estimate)*: software-only BYO-camera **B2C USD 60?180/yr**, **B2B
academy/club USD 300?1,500/yr** per location (undercuts hardware-bundled Veo/Pixellot/
Spiideo; comparable to SwingVision software-only).

PMF risks (from the readiness review)
- **Capture quality is the gate.** Accuracy is capture-conditional; broadcast/handheld
  footage is a failure mode. The wedge (static, tripod, full-court) is defensible but
  narrows the addressable market ? enforce/guide it (C11, `VIDEO_RECORDING_GUIDELINES`).
- **Value beyond raw segments is table stakes.** Time-saved editing alone is weak;
  per-rally tags, stats, and a correction loop are what make it sticky (C10).
- **Golden labels are the recurring blocker** across the offline segmenter, audio
  fusion, and measurable quality ? the highest-leverage investment (C8).

Refined priority order (quality/data phase ? "focus on quality once new videos
land")
Carried from the readiness review; commercialization items above are tracked in
parallel.

1. **Golden-set expansion + data-flywheel UX** (highest with new labeled videos):
   ingest the new batch, expand calibration/CV/regression baselines, make label
   contribution + safe retraining easy and non-contaminating (eval/training separation).
2. **Hybrid unification + observability hooks** (deferred item 7, now higher): extract
   shared logic from `tracknet_hybrid`/`wasb_hybrid` before heavy AI timing/metrics
land.
3. **Detector/windowing quality iteration** with new data: re-tune velocity/merge/
   min-duration + `rally_gate`; validate WASB on real video; trial audio fusion
(ADR-007).
4. **Deeper value layer** (beyond minimal exports, C10): per-rally tagging/notes ?
   `candidate_segments`, per-player/match stats, clip gallery, human-correction loop.
5. **Input-quality enforcement & guidance polish** (C11): more guideline dimensions ?

```
178        validators/report rules; "your video is good enough" UX.
179     6. **Multi-sport maturation + recording best practices.**
180     7. **SaaS / multi-user / async foundations** (C12) ? lower until monetization is firm.
181     8. **Polish & distribution** (C9): installer story, first-run path, codec/output,
economics.
182
183     **Recommendation:** with the mid-week labeled-video batch, spend ~80% on #1 + #3
184     (flywheel + quality iteration on real data), then unification (#2) and deeper value
(#4).
185
186     ---
187
188     ## 7. Open questions needing owner / counsel sign-off ??
189
190     1. **Ultralytics Enterprise price** ? only relevant if a permissive detector ever proves
191        insufficient (fallback). Quote-only.
192     2. **Per-checkpoint / dataset licenses** for WASB / TrackNetV3 / any VLM ? train-own or
193        confirm each (C3).
194     3. **Exact Via LA (AVC) / Access Advance (HEVC) rates & free thresholds** ? confirm live
195        figures with counsel before relying on the "small SaaS under threshold" reading
(C5/C6).
196     4. **Real per-video GPU cost & latency** of self-hosted VLM vs Gemini paid at expected
197        throughput (C7).
198     5. **Provision Ada-class GPU (L4/L40)** for HW AV1 encode? (C5 ? gates the codec
switch.)
199
200     ---
201
202     ## Changelog
203
204     - **2026-06-07** ? Added the market-research-of-record (`PMF_MARKET_RESEARCH.md`):
205       ranked beachhead (India badminton academies #1), pricing benchmarks, and a C13
206       "beachhead validation" workstream. Wired the summary into §6.
207     - **2026-06-07** ? Created this living tracker. Merged the 2026-05-30 monetization
208       audit + the commercial-readiness review. **Refreshed since the audit:** C1 AGPL
209       removal now ? shipped (was the #1 blocker); added C5 as a ? (output is still H.264,
210       the audit's AV1 recommendation is not yet implemented); C9 packaging tied to
211       `DEFERRED_HARDENING_PLAN.md`; reflected PR #50/#51 (input-quality reports, value-layer
212       exports). Archived `COMMERCIAL_READINESS_REVIEW.md`.
213
214     ---
215
216     ## See also
217     - `PMF_MARKET_RESEARCH.md` ? market-study-of-record (sizing, segments, competitors,
beachhead; cited).
218     - `archives/MONETIZATION_AUDIT.md` ? licensing/economics research-of-record (sources +
verification transcript).
219     - `archives/COMMERCIAL_READINESS_REVIEW.md` ? superseded assessment (history).
220     - `archives/decisions/DECISIONS.md` ? ADR-005 (permissive stack), ADR-006 (vendor
process).
221     - `ROADMAP.md` ? Phase-4 offline segmenter (the C4 "(A) eliminate" path).
222     - `DEFERRED_HARDENING_PLAN.md` ? packaging (C9), async (C12) designs (pending approval).
223     - `GOLDEN_SET_VENDORING.md` ? vendor process + Roora pilot (C8).
224     - `BACKLOG.md` ? `supervision` migration (C2), `obsreport` pin, codec follow-ups.
225
```

6. assistant (2026-06-15T14:57:58.284Z)

Let me read the remaining cross-reference docs to validate naming, framing, capture rules, and watermark text.

7. user (2026-06-15T14:57:59.269Z)

```
1       # Field & Product Outreach Associate ? KhelSutra (Sports-Tech, Bengaluru)
2
```

Help a Bengaluru sports-AI product meet its first real users.

Duration	~3?4 weeks, part-time (flexible; mostly afternoons/evenings when academies are active)
Location	Bengaluru ? in-person visits to badminton academies
Pay	? ___ per completed academy visit + travel reimbursement; recording sessions (stretch goal below) paid extra ? ? ___ per session
Reports to	Avi (phone/WhatsApp: ___)

The product, in plain words

KhelSutra turns ordinary practice/match video from a badminton academy into ready-to-use pieces: it finds the rallies in the recording automatically, cuts **highlight reels**, and produces a **simple summary** (how many rallies, how much actual play time). It works on video from a phone on a tripod ? no special cameras, no hardware to install. We're building a private web app that a few invited academies will be able to try.

The job, in one paragraph

Visit good badminton academies across Bengaluru and have honest conversations with **coaches AND students**: how do they work with video today, would something like this actually help them, and **what would make it genuinely useful for them** ? including what's missing or wrong with the idea. You work from a provided conversation guide, write down what people *actually say* (the negative answers matter most), and debrief with Avi after each visit. **You are gathering feedback, not selling.** No tech background needed.

The stretch goal (extra pay)

For academies that are interested, you coordinate a simple recording session ? a phone on a tripod, one-page checklist provided. We run that video through KhelSutra, and you go back and show them **their own court, their own players, as highlight reels and a report**: *"this is what it looks like for you."* Then you write down exactly what they say. There is no stronger feedback than that moment.

What you'll actually do

- Visit **10?15 academies** (intro + conversation, ~20?30 minutes each).
- Talk to the **owner/head coach** first, and ? with their permission ? a few **adult (18+) students** about whether they'd watch/share their own rally clips and what they'd want from them.
- Fill **one structured answer sheet per visit** (we provide it) ? their words, not yours, including the most negative thing said.
- Show a short sample reel on your phone **only at the point the guide says to** (after the open questions ? it changes people's answers if shown early).
- Debrief with Avi after each visit (WhatsApp/call). **Your first two visits are done together with him**, so you're never improvising.
- *Stretch:* coordinate recording sessions and present the sample results back.

You're a good fit if

- You're comfortable walking into an academy and starting a conversation with a stranger ? and just as comfortable hearing "no, this is useless to us."
- You can capture what people said **accurately** ? quotes, not summaries.
- Kannada / Hindi / English, as the conversation needs (you choose the mix).
- You play or follow a racket sport (helps a lot; not required).
- No technical skills needed. Reliability matters more than anything else.

What you get

```
66      - Paid per completed honest visit ? **not** per positive answer. A well-documented
67        "they hated it, here's why" is a fully paid, fully successful visit.
68      - Ground-floor view of a real sports-AI product finding its market; your field
69        notes directly shape what gets built next.
70      - Flexible hours, short commitment, clear scope.
71
72      ## Ground rules (the short version ? full version provided on joining)
73
74      1. Honesty over positivity. Never oversell, never promise features, prices, or dates.
75      2. **No recording of any person without written consent** ? and sessions involving
76         **minors are never recorded** for our use, period. In regular visits you record
77         nothing at all: notes only.
78      3. People's contact details stay between you and Avi.
79
80      ## How to apply
81
82      Message Avi at ___ with **"FIELD"**, one line about a sport you follow, and one
83      line about a time you had to get honest feedback out of someone. Next step: a
84      15-minute phone call with Avi.
85
```

8. user (2026-06-15T14:57:59.670Z)

```
1       # Field Visit Playbook ? academy conversations (KhelSutra)
2
3       > **Recruiting?** Hand candidates
[`PRODUCT_FIELD_ASSOCIATE_FLYER.md`](PRODUCT_FIELD_ASSOCIATE_FLYER.md)
4       > ? that's the role description. **This document is the working playbook the hired
5       > associate gets on day one:** the conversation script, the ground rules, and the
6       > Phase-2 (recording / show-back) procedure.
7
8       **Duration:** ~3?4 weeks, part-time (flexible hours, mostly afternoons/evenings when
academies are active)
9       **Location:** Bengaluru ? in-person visits to badminton academies
10      **Compensation:** ? ___ per completed academy visit + travel reimbursement; Phase-2
sessions paid extra ? ? ___ per session *(rates filled at hire; same numbers as the recruiting
flyer)*
11      **Reports to:** Avi (phone/WhatsApp: ___ )
12
13      ---
14
15      ## The gig in one paragraph
16
17      KhelSutra takes ordinary practice/match video from a badminton academy and finds the
rallies automatically, builds
18      highlight reels, and generates a simple summary (rallies found, actual play time); we're
building a private web app
19      that a few invited academies will be able to try. We need honest, on-the-ground feedback
from real academies ? **how
20      they work with video, whether this actually helps them, and what would make it genuinely
useful**. **Your job: visit
21      10?15 good academies, have a friendly 20?30 minute conversation with the owner/head
coach (and, with permission, a
22      few adult (18+) students) using the script below, and write down what they actually
say.** No selling. No tech knowledge needed. For interested academies there's a Phase-2 follow-
up
23      (a recording session ? then you return and show them THEIR OWN footage as highlights)
with additional compensation.
24
25      ---
26
27      ## Phase 1 ? The conversation (what you actually do)
28
29      **Who to talk to:** the academy owner or head coach ? the person who decides what the
academy spends money on.
```

If you get a junior coach, be friendly, gather what you can, and ask when the owner is available.

Opening (keep it this honest):
> "Hi, I'm helping a Bengaluru engineer who's building an AI tool for badminton academies ? it takes normal practice
> video and automatically cuts it into rallies and highlights. He asked me to talk to a few serious academies to
> understand how you work with video today. Could I ask you a few questions? About 15?20 minutes, and we're not
> selling anything today."

The questions (ask in this order; the starred ones matter most):

1. Do you ever video-record sessions or matches here? How ? phone, tripod, CCTV, a hired person?
2. ? **Do you have recorded footage sitting around that nobody has time to watch?** (multi-court hall recordings,
   tournament days, CCTV) ? roughly how much?
3. Who would watch rally clips or highlights if they existed ? coaches for training? players? parents?
4. What do you use today for any of this? (Listen for: Machaxi, VISIST, Game Theory, Stupa, BadPro+, just
   YouTube/phone.) What do you like / what's missing in it?
5. ? **If you could hand over a full session's video and get back the rallies cut out + a highlight reel + a simple
   summary ? would you pay per match for that? What would that be worth to you, per match?** Then **stop talking and
   wait** ? let THEM name the number. If they genuinely won't after a pause and one "roughly?", you may offer the
   ladder: "?100, ?300, ?500?" **On the sheet, tick whether their number came BEFORE or AFTER you offered the
   ladder** ? it changes what the answer means. (Note their face, not just the words.)
6. Your hall has multiple courts running at once, right? Does that make video less useful today?

? NOW (and only now) show the sample reel ? a ~60-second real highlight reel we put on your phone. Say: *"this is
what it produces ? the rallies found and cut automatically."* Then shut up and watch their face. Write down their
FIRST reaction verbatim, and ask one follow-up: **"what would this need to do for it to be useful for YOUR academy?"**
(Improvement asks, in their words, go on the sheet. Showing it earlier anchors every answer above ? never lead with it.)

7. Would you let us record one session here (we pay a small fee for the time) and come back showing what the tool
   produces on YOUR court, your players? *(This is the Phase-2 door ? just note yes/no/maybe and who to call.)*

Students (with the coach's permission ? adult (18+) students ONLY, 5 minutes): ask the coach to point out which
students are 18 or older ? "senior batch" players who are minors do NOT count. Then ask 2?3 of them:
(a) *"Would you watch clips of your own rallies if they existed after each session?"* (b) *"Would you share them ?
with whom?"* (c) *"What would you want them to show?"* Capture their answers in the sheet's student block. Don't
collect students' contact details; if no adult students are available, skip the student questions ? note that instead.

**After each visit, fill ONE answer sheet ? the template is [`FIELD_VISIT_ANSWER_SHEET.md`](FIELD_VISIT_ANSWER_SHEET.md)
(print one per visit):** academy name, location, # courts, person spoken to + role + contact, answers 1?7 (their

words, not yours), the most negative thing they said (required), exact quotes, your gut read, and whether they're a
Phase-2 candidate. *(How the answers get scored and what totals trigger which decision is our job, not yours ?
just capture honestly.)*

Ground rules & gotchas (read twice ? this is the difference between useful and useless)

1. **You are researching, not selling.** If they ask "how much will it cost", say "that's exactly what he's trying to
   figure out ? what would feel fair to you?" Never quote a price yourself.
2. **Past behavior beats opinions.** "Would you use video?" gets polite lies. "When did you LAST record a session, and
   what happened to that file?" gets the truth. Prefer the second kind of question.
3. **Don't promise anything.** No features, no dates, no free trials. You may SHOW the standard sample reel (it's
   real output ? at the scripted moment only, after Q6) and you may offer the Phase-2 idea (a paid recording session
   + free sample output of their own court), and only as "would you be open to it". Nothing else.
4. **Write down disappointing answers verbatim.** A "no, we'd never pay for that" with a reason is worth MORE to us
   than five vague yeses. You're paid per completed honest visit, not per positive answer.
5. **If they already use a competitor (Machaxi/VISIST/Game Theory etc.) ? goldmine.** Ask what it does well, what it
   doesn't, and what they pay. Spend extra time here.
6. **Multi-court is our favorite topic.** If they complain that video is useless because "all the courts are in frame
   and it's chaos" ? capture that complaint in their exact words. That's the problem we solve.
7. **No recordings of people without clear permission.** In Phase 1 you record NOTHING ? notes only. Don't photograph
   the academy or players unless the owner offers.
8. **Don't oversit.** 20 minutes, thank them, leave a good impression ? we may come back to record.
9. **Confidentiality:** don't describe the tech beyond the opening line, don't name other academies' answers, and
   don't share the answer sheets with anyone except Avi.

Phase 2 (only for academies that said yes ? separate briefing & pay)

You coordinate a recording session: we pay the academy a small fee, set up a phone/camera on a tripod (we provide a
one-page capture checklist: behind the court, elevated, whole court in frame, landscape, 1080p/60fps, don't move it),
record 1?2 sessions ? then we process the footage through KhelSutra and **you return to present the academy its own
sample output** (their court, their players, as a highlight reel + summary, free) and fill a short reaction sheet:
what they said when they saw it, what they'd change, whether they asked to keep using it. That show-back conversation
is the single most valuable artifact this whole engagement produces. **If they ask "can we keep using it?"** ? that's
the best possible signal: write it down verbatim, say *"I'll pass that straight to Avi"*, and promise nothing. **Hard rules:** written consent from the academy
AND the adult players being recorded; **sessions with minors are not recorded for our library** ? adult sessions only;
the academy gets told plainly what the footage is for (building and demonstrating the tool).

```
109
110     ---
111
112     ## Day one
113
114     Your first 2 visits are paired (you + Avi, or a live phone walkthrough) so you're never
improvising. Before visit 1:
115     read this playbook twice, have the sample reel saved offline on your phone (Avi provides
the ~60-second trim), and carry printed answer sheets
116     ([`FIELD_VISIT_ANSWER_SHEET.md`](FIELD_VISIT_ANSWER_SHEET.md) ? one per academy).
117
```

9. assistant (2026-06-15T14:58:03.061Z)

Now let me check the README and a couple of supporting docs to validate the watermark text and the "Generated by" naming, plus confirm capture/recording guidelines.

10. user (2026-06-15T14:58:04.234Z)

```
1       # Badminton Highlight & Rally Indexer
2
3       A local, AI-powered web application and CLI utility designed to process long-form
badminton recordings (e.g., from GoPro), run comprehensive pluggable validations (quality,
integrity, and relevance checks), index all rallies into a relational SQLite database, and
dynamically compile custom highlight reels using high-speed FFmpeg stitching.
4
5       The pipeline is built around three pluggable registries ? **validators**,
**segmenters**, and **AI providers** ? plus a **sport profile** registry so the same engine can
target other racket/court sports in the future. Badminton is the default profile.
6
7       > **Where this is heading:** the project is evolving from an AI-API-dependent prototype
into a self-improving tool that learns rally detection *offline* from a commercially-licensed,
annotated corpus. See **[docs/ROADMAP.md](docs/ROADMAP.md)** for the four-phase plan and
**[docs/archives/decisions/DECISIONS.md](docs/archives/decisions/DECISIONS.md)** (historical
ADRs) for the rationale.
8
9       ---
10
11      ## ? Key Features
12
13      *   **Pluggable Video Validation Framework**: Easily extendable base-class architecture
that automatically runs checks on:
14          *   *Static Format & Resolution*: Verifies container is MP4, resolution >= 720p, and
framerate >= 30 FPS.
15          *   *PTS Timeline Continuity*: Scans keyframe timestamps for timeline gaps, jumps,
or editing splices to detect tampering or splices.
16          *   *Scene Cut Density Check*: Measures sub-sampled HSV frame histogram correlation
to verify visual cuts. A continuous raw camera stream should have near 0 cuts.
17          *   *Blurriness Check*: Calculates average frame Laplacian variance to reject out-
of-focus footage.
18          *   *Relevance (Is it Badminton?)*: Uses high-performance HSV color court matching
to identify court layouts.
19      *   **Six Pluggable Rally Segmenters** (`backend/pipeline/segmenters/` ? `motion`,
`gemini`, `yolo_hybrid`, `tracknet_hybrid`, `wasb_hybrid`, `fusion_hybrid`). The three core
ones:
20          *   `yolo_hybrid` ? two-stage pipeline. Stage 1 runs **torchvision person
detection** (Faster R-CNN, BSD-licensed) with **ByteTrack** tracking (supervision, MIT-licensed)
to cheaply filter the video down to a small set of high-motion candidate windows. Stage 2 hands
each candidate (with a small padding) to the AI provider for high-precision semantic boundaries
and shot counts. Massively cheaper than running the LLM on the full video. *(The registry key
remains `yolo_hybrid` for back-compat; the AGPL-licensed YOLOv8 was replaced with permissive
components ? see [docs/archives/MONETIZATION_AUDIT.md](docs/archives/MONETIZATION_AUDIT.md) and
ADR-005.)*
21          *   `gemini` ? sends the entire video (or a `chunk_duration` slice) to Google Gemini
directly. Highest precision per call, highest cost.
```

```
22          *   `motion` ? zero-dependency OpenCV pixel-difference heuristic. No API key
required. Useful as a baseline or for offline/air-gapped use.
23      *   **Pluggable AI Provider Registry** (`backend/providers/`): The Gemini provider ships
in-tree; new providers (e.g. an OpenAI or local-model backend) can be registered without
touching the segmenters.
24      *   **Pluggable Sport Profile Registry** (`backend/sports/`): The per-sport LLM prompt
and relevance config live in a `SportProfile`. Add a new sport by subclassing `SportProfile` and
registering it.
25      *   **SQLite Indexing**: Each processed video is keyed by MD5 hash. Play segments
(start/end/duration/server/receiver/metadata) and per-segment events are persisted in
`output/sports_indexer.db`. Re-running on the same file is a cache hit.
26      *   **Query System**: Filter indexed play segments by `server`, `receiver`,
`duration_min`, and `event_type` (the schema supports rich event types like
smashes/serves/fouls; today the shipped segmenters mainly populate timing + estimated
`shot_count`).
27      *   **Zero-Dependency Local UI**: Served statically directly by FastAPI (Vite + Node.js
NOT required!) featuring a glassmorphic dashboard, responsive filtering, and immediate clip
stitching.
28      *   **High-Speed FFmpeg Stitching** with configurable `begin_padding`, `end_padding`,
and `min_shot_count` filtering, plus an optional **branding watermark** burned into every reel
(**on by default** ? `"Generated by Khelsutra.guru"`, fully configurable, with an off switch).
29      *   **Detailed CLI Debugging Tools**: `--debug-clip <t>` inspects frame metrics,
validator scores, and motion values around a timestamp. `--trim-start/--trim-end` extracts an
arbitrary segment via stream-copy ffmpeg for fast iteration.
30
31      ---
32
33      ## ?? Setup & Diagnostics
34
35      ### 1. External System Requirements
36      The indexer depends on **FFmpeg** and **FFprobe** to parse media containers, extract
keyframes, and stitch clips.
37
38      *   **On Windows (PowerShell as Admin)**:
39          ```powershell
40          winget install Gyan.FFmpeg
41          ```
42          *Restart your terminal afterwards to register the path changes.*
43
44      ### 2. Install the package
45
46      The project is packaged with PEP 621 metadata in `pyproject.toml` (hatchling
47      build backend). Install it editable:
48
49      ```bash
50      # Runtime deps only:
51      pip install -e .
52      # Plus test + lint/type tooling (pytest, pytest-cov, httpx, ruff, mypy):
53      pip install -e .[dev]
54      ```
55
56      This registers the `indexer` console entry point (so `indexer --server`,
57      `indexer --help`, etc. work) and puts the `backend` and `training` packages on
58      your path.
59
60      > **PyTorch is installed separately.** `torch`/`torchvision` are intentionally
61      > *not* in `pyproject.toml`: their CPU/CUDA wheels need an out-of-band wheel index
62      > that PEP 621 cannot express. Install them yourself, matching your hardware:
63      > ```bash
64      > # NVIDIA GPU (CUDA 12.4):
65      > pip install "torch>=2.12.0" "torchvision>=0.27.0" --index-url
https://download.pytorch.org/whl/cu124
66      > # CPU only:
67      > pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
68      > ```
```

3. Diagnostics Checker (`scripts/doctor.py`)
Run the diagnostics checker to verify your system dependencies, initialize a default config, and (optionally) bootstrap the Python packages:
```bash
python scripts/doctor.py
# or, via the CLI:
indexer --setup
```
This script will:
*   Validate `ffmpeg` and `ffprobe` are present in your PATH.
*   Initialize a default `config.json` (if one does not exist) with thresholds for blur, court HSV bounds, motion signals, YOLO velocity, chunking, and stitching.
*   Detect whether an NVIDIA GPU is available (via `nvidia-smi`) and select the matching PyTorch wheel index automatically (`cu124` if GPU present, `cpu` otherwise).
*   Install everything from `requirements.txt`: `fastapi`, `uvicorn`, `opencv-python`, `numpy`, `pydantic`, `jinja2`, `python-dotenv`, `google-genai`, `torch`, `torchvision`, `supervision`.

> The default `yolo_hybrid` segmenter uses **torchvision**'s COCO-pretrained Faster R-CNN, whose weights are downloaded and cached automatically by torchvision on first run ? nothing is bundled in the repo. (Pick a lighter/heavier detector via `indexing.detector_model`: `fasterrcnn_resnet50_fpn` (default), `fasterrcnn_mobilenet_v3_large_fpn`, or `retinanet_resnet50_fpn`.)

? Running the Application

1. Launching the Local Web App Dashboard
To start the FastAPI server and open the web dashboard:
```bash
python -m backend.main --server --port 8000
```
Open your browser and navigate to **http://localhost:8000**.
Simply input your local raw video file path in the sidebar to process, validate, index, and compile highlights instantly.

2. Running in CLI Processing Mode
To process and index a video completely via terminal commands:
```bash
python -m backend.main --video-path "C:/path/to/my_video.mp4"
```
This outputs a validation summary and indexes all rallies directly into the local SQLite database, using whichever segmenter is set as `default_segmenter` in `config.json`. **The shipped `config.json` sets `gemini`** (needs `GEMINI_API_KEY`; without it the run silently produces 0 segments and the observability report flags `silent_provider_noop`). Set it to `yolo_hybrid`/`wasb_hybrid`/`motion` for the local paths.

To explicitly pick a segmenter for one run:
```bash
python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter yolo_hybrid
python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter gemini
python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter motion
```

3. Rich CLI Debugging (`--debug-clip`)
To diagnose exactly why a video segment was registered as active/dead play, or inspect detailed focus/continuity metrics at a particular time in seconds:
```bash
python -m backend.main --video-path "C:/path/to/my_video.mp4" --debug-clip 125.5
```
Outputs granular validation check logs and motion ratios within a +/- 3 second frame window around the timestamp.

```
    ### 4. Extracting a Trimmed Segment (`--trim-start` / `--trim-end`) ? also the quickest
watermark preview
    Extract a slice of a recording to disk (re-encoded; and, with the default-on branding
watermark, stamped **"Generated by Khelsutra.guru"** in the corner):
    ```bash
    python -m backend.main --video-path "C:/path/to/my_video.mp4" \
        --trim-start 600 --trim-end 660 --trim-output slice_10m_to_11m.mp4
    ```
    Output lands in `--output-dir` (default `output/`) unless `--trim-output` is an absolute
path. This is the fastest way to *see* the watermark on a real clip ? set
`stitching.watermark.enabled: false` in `config.json` to turn it off.

    > To preview the exact overlay on any file without the pipeline:
    > ```bash
    > ffmpeg -i in.mp4 -vf "drawtext=text='Generated by Khelsutra.guru':font='DejaVu Sans':f
ontcolor=white@0.85:fontsize=h/28:x=w-tw-24:y=h-th-24:box=1:boxcolor=black@0.4:boxborderw=10"
-c:a copy out.mp4
    > ```

    ### 5. End-to-End "Process + Stitch" Script
    `scripts/run_process_and_stitch.py` is an interactive convenience runner: it MD5-hashes
a hardcoded video path, reuses cached indexed segments if available (offering to re-index from
scratch), and then prompts for stitching parameters (`begin_padding`, `end_padding`,
`min_shot_count`) before compiling the final highlight reel. Useful when iterating on stitching
settings against a fixed large recording without re-running expensive segmentation.

    ### 6. Cache Hygiene & Disk Cleanup
    The trajectory detectors (`wasb_hybrid` / `tracknet_hybrid`) stage a copy of each video
into WSL and decode every frame to PNG ? easily **tens of GBs per 4K video**. To keep disk usage
bounded:

    *   **Automatic self-clean:** after a *successful* run the frame cache + staged copy are
deleted (`indexing.{wasb,tracknet}.keep_frames`, default `false`). On failure they're kept so a
re-run resumes from the checkpointed detector cache.
    *   **Manual sweep (dry-run-first):** `tools/cleanup_caches.py` reports and (with
`--apply`) reclaims what failed/old runs left behind, across both the WSL stage dir and
`output/`:
        ```bash
        python -m tools.cleanup_caches                  # report what's reclaimable (safe;
deletes nothing)
        python -m tools.cleanup_caches --apply          # delete the regenerable caches
        python -m tools.cleanup_caches --keep-video GX010125   # protect one video's cache
(id or filename stem)
        python -m tools.cleanup_caches --older-than 7   # only purge entries older than 7
days
        ```
        It treats frame caches, staged/proxy videos, and handoff/temp dirs as
regenerable (purgeable) and always keeps trajectory CSVs, the SQLite DB, and final reels.
Add `--include-wasbcache` to also drop detector resume caches. `--summary` prints a one-line
nudge (used by the optional SessionStart hook).

    ## ?? Threshold Tuning (`config.json`)
    You can adjust the parameters of your validators, segmenters, AI provider, and stitcher
by editing `config.json` in the root folder. The default config emitted by `scripts/doctor.py`:

    ```json
    {
        "sport": "badminton",
        "ai_provider": "gemini",
        "ai_model": "gemini-flash-latest",
        "validation": {
            "min_resolution_width": 1280,
            "min_resolution_height": 720,
```
```

```
            "min_fps": 29.9,
            "min_blur_threshold": 20.0,
            "max_scene_cuts_per_minute": 0.5,
            "relevance": {
                "court_green_lower": [35, 40, 40],
                "court_green_upper": [85, 255, 255],
                "court_pixels_min_ratio": 0.05
            }
        },
        "indexing": {
            "default_segmenter": "gemini",
            "use_gpu": true,
            "motion_threshold": 0.08,
            "min_segment_duration": 3.0,
            "dead_time_merge_gap": 2.0,
            "chunk_duration": 90,
            "chunk_overlap": 10,
            "yolo_velocity_threshold": 0.15,
            "yolo_frame_skip": 3,
            "yolo_tracker": "bytetrack.yaml",
            "yolo_prefetch_workers": 2,
            "min_action_window_duration": 2.0,
            "ai_handoff_padding": 2.0,
            "ai_max_workers": 5
        },
        "output": {
            "default_highlights_name": "highlights.mp4",
            "db_name": "sports_indexer.db"
        },
        "stitching": {
            "begin_padding": 0.5,
            "end_padding": 0.5,
            "use_cache": false,
            "min_shot_count": 1,
            "watermark": {
                "enabled": true,
                "text": "Generated by Khelsutra.guru",
                "position": "bottom-right",
                "opacity": 0.85,
                "logo_path": "",
                "font_path": ""
            }
        }
    }
    ```

    Notable knobs:
    *   `validation.min_blur_threshold` ? defaults to `20.0` (deliberately permissive for noisy indoor-arena GoPro footage; see `docs/archives/TEST_RUN_SUMMARY.md` for why).
    *   `indexing.default_segmenter` ? `yolo_hybrid` | `gemini` | `motion`.
    *   `indexing.use_gpu` ? when true and CUDA is available, YOLO runs on GPU.
    *   `indexing.yolo_velocity_threshold` ? fraction of frame width per second; lower values catch slower play, higher values are stricter about what counts as "action".
    *   `indexing.yolo_frame_skip` ? process every Nth frame during YOLO tracking. `3` at 30 fps ? 10 samples/sec.
    *   `indexing.chunk_duration` / `chunk_overlap` ? how to split long videos for the YOLO phase.
    *   `indexing.ai_handoff_padding` ? seconds of slack added around each YOLO candidate window before sending to the AI provider.
    *   `indexing.ai_max_workers` ? parallel calls to the AI provider during phase 3.
    *   `stitching.min_shot_count` ? drop play segments whose estimated shot count is below this threshold when compiling highlights. **Enforced on both the CLI runner and the `/api/compile` endpoint** (segments that carry no `shot_count` metadata are exempt, so explicitly-requested segment ids can't silently vanish under the default floor).
    *   `indexing.wasb.keep_frames` / `indexing.tracknet.keep_frames` ? after a
```
```

successful trajectory-detector run, the decoded frame cache and the staged video copy (the dominant disk consumer ? ~GBs per 4K video) are deleted automatically. Default `false` (= auto-delete); set `true` only for debugging. On **failure** they're always kept so a re-run resumes. `indexing.{wasb,tracknet}.min_free_gb` warns before a run if WSL free space is below the threshold (0 = off). See **§ Cache hygiene** below and `python -m tools.cleanup_caches`.
218   *   `stitching.watermark` ? branding overlay burned into every highlight clip (**on by default**). `enabled: false` turns it off; `text` is the fully-configurable string; `position` ? {`bottom-right`, `bottom-left`, `top-right`, `top-left`}; `opacity` 0?1; optional `logo_path` (transparent PNG). It ships with a **bundled Apache-2.0 font (Roboto Slab)** so the text renders out-of-the-box **without a system fontconfig setup** (which Windows ffmpeg often lacks); set **`font_path`** to a different `.ttf` (your own, or another in `backend/assets/fonts/`) to change the typeface. If the watermark encode still fails, the clip is re-encoded without it (so the reel is never lost) and the ffmpeg reason is logged.
219
220   ---
221
222   ## ? Frequently Asked Questions (FAQ)
223
224   ### Q1: The server complains that `ffprobe` is not found.
225   Ensure FFmpeg is installed and added to your system environment variables. You can check this by typing `ffmpeg -version` or `ffprobe -version` in your terminal. If running `scripts/doctor.py` outputs a `[FAIL]`, re-install FFmpeg or manually copy the binary files to a PATH folder.
226
227   ### Q2: Why does my video fail the Relevance check?
228   Our default relevance check scans for the typical indoor green badminton court floor color space (HSV). If your court is blue, or uses standard wood grain flooring, simply adjust the HSV values inside your `config.json` under `validation.relevance.court_green_lower` and `court_green_upper` to match your court, or lower the `court_pixels_min_ratio` to `0.0` to bypass.
229
230   ### Q3: How do I add a new custom validator to the pipeline?
231   1. Create a new python file in `backend/pipeline/validators/` (e.g. `my_check.py`).
232   2. Subclass `VideoValidator` and implement `validate(self, video_path, config, context)`.
233   3. Register your class inside `backend/pipeline/validators/__init__.py` using `registry.register(MyCheckValidator())`.
234   It's that simple! The app will automatically run it, display it in the CLI diagnostics summary, and render it in the frontend checklist.
235
236   ### Q4: Why does the CLI or server output freeze or take a long time on large videos?
237   *   **Print Buffering**: Standard Python buffers console output when stdout is redirected or running in background shells on Windows. To get real-time unbuffered log statements immediately, run commands with Python's unbuffered flag:
238   ```bash
239   python -u -m backend.main --video-path "C:/path/to/video.mp4"
240   ```
241   *   **Video Seeking Overhead**: Seeking to random frames in high-bitrate long H.265/HEVC videos using OpenCV's `cap.set(cv2.CAP_PROP_POS_FRAMES)` incurs massive I/O decoding delays. The `motion` segmenter reads sequentially and skips frame decodes using container-level `cap.grab()`, which delivers a **100x speedup** on multi-gigabyte raw files. `yolo_hybrid` sidesteps the problem entirely by extracting fixed chunks via ffmpeg first.
242
243   ### Q5: Why did the video fail with "No keyframes / packet timestamps could be extracted"?
244   Some H.265/HEVC streams (e.g. GoPro footage) encode frames as GDR or CRA keyframes. Standard FFmpeg frame-level skipping (`-skip_frame nokey`) fails to identify them, returning empty logs. We resolved this by extracting packet indexes (`packet=pts_time,flags`) at the container demuxer level instead, which is incredibly robust and executes instantly without decoding a single pixel.
245
246   ### Q6: How can I quickly test the indexing without waiting to parse an entire multi-GB video file?
247   You can pass the `--max-frames <N>` argument in CLI or `max_frames` in the POST API payload. This restricts the segmenter frame loop to process only the first N sub-sampled

frames, enabling you to test motion filters and court relevance checks in seconds:
    ```bash
    python -m backend.main --video-path "C:\Users\avidu\Videos\LargeTestVideo.MP4" --max-frames 300
    ```
    (At 5 sub-sampled frames per second, `--max-frames 300` analyzes exactly the first 60 seconds of video).

    ### Q7: How do I configure and run an AI-powered segmenter (Gemini / YOLO-hybrid)?
    Both `yolo_hybrid` (default) and `gemini` need an API key for the AI provider. To use Google Gemini:
    1. Create a local `.env` file in the project root (this is already in `.gitignore` to prevent secret leaks).
    2. Set your Google AI Studio API key in the file:
       ```env
       GEMINI_API_KEY=your_key_here
       ```
    3. Run with whichever segmenter you want:
       ```bash
       # Default: cheap candidate filtering with YOLO, then precise boundaries via Gemini
       python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter yolo_hybrid

       # Full-video Gemini analysis (more expensive, highest semantic precision per call)
       python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter gemini

       # Pure CV baseline ? no API key needed
       python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter motion
       ```
       Or set `"default_segmenter": "yolo_hybrid"` (or `"gemini"`/`"motion"`) under `"indexing"` in `config.json`.

    The AI provider used by `yolo_hybrid` and `gemini` is controlled by `ai_provider` + `ai_model` at the top level of `config.json`. Today only `"gemini"` is registered; new providers can be added under `backend/providers/`.

    Trajectory-based segmenters (`wasb_hybrid`, `tracknet_hybrid`). The registry also includes two shuttle-trajectory variants. Like `yolo_hybrid`, they cheaply filter the video down to candidate windows *locally* and then route each window to the AI provider (`gemini`) for precise boundaries ? so they need the same `GEMINI_API_KEY`. **If an observability report shows `segmenter_actual: wasb_hybrid` (or `tracknet_hybrid`), that is a valid, registered segmenter ? not an error.** Pick one with `--segmenter wasb_hybrid` or via `default_segmenter`. (`wasb_hybrid` additionally requires the WSL WASB setup ? see docs/TRACKNET_WSL_SETUP.md.)

    ### Q8: How does the `yolo_hybrid` pipeline actually save money?
    Sending a multi-gigabyte video to a hosted LLM is slow and expensive. `yolo_hybrid` runs in four phases:

    1. **Chunking** ? splits the video into overlapping windows (`chunk_duration`, `chunk_overlap`) so we can process incrementally.
    2. **Detection + tracking** ? torchvision person detection (Faster R-CNN) + ByteTrack runs on each chunk locally (GPU-accelerated when available), measuring per-track velocity. Frames whose normalised player velocity exceeds `yolo_velocity_threshold` are kept; the rest are dropped. Adjacent active frames are merged into candidate windows, with sub-`min_action_window_duration` windows discarded.
    3. **AI handoff** ? only the surviving candidate windows (padded by `ai_handoff_padding` seconds on each side) are extracted and sent to the AI provider for precise rally boundaries and shot-count estimation. These calls run in parallel up to `ai_max_workers`.
    4. **DB population** ? confirmed segments are written to `play_segments` along with `shot_count`, `shot_count_confidence`, and a `detector` tag like `yolo-hybrid-gemini-2.5-flash`.

    In practice this typically cuts AI-provider runtime/cost by an order of magnitude versus uploading the full video to `gemini` directly, while preserving the LLM's strength at semantic boundary detection.

Q9: How can I debug/test the Gemini Segmenter on a small portion of a video without uploading a huge multi-GB file?

Ingesting and uploading multi-gigabyte video files to Google Cloud can be slow and expensive. We built a native **fast local trimming** developer debugging feature:

1. Open your `config.json` file.
2. Under the `"indexing"` configuration block, add a `"debug_duration"` key (in seconds):

    ```json
    "indexing": {
        "default_segmenter": "gemini",
        "debug_duration": 15.0,
        ...
    }
    ```

3. When `"debug_duration"` is set, the Gemini segmenter will instantly run a zero-re-encode, sub-second `ffmpeg` slice to create a tiny temporary clip of the first 15 seconds, upload only that tiny clip to Google AI Studio (which takes <1 second to upload and <5 seconds to process), run the model query, and clean up the temporary files automatically. This allows you to iterate on model prompts, evaluate validation overlaps, and verify rally boundaries in under 10 seconds!

Q10: Can I trust the per-segment metadata from the `motion` segmenter?

No. The zero-dependency `motion` segmenter detects *timing* via pixel-difference, but its per-segment **metadata is heuristic/synthetic** ? `intensity`, `avg_speed_kmh`, the serve/smash/foul/winner **events**, and `server`/`receiver` are generated values, **not measured**. They exist so the schema is populated for demos; do not treat them as ground truth. For real shot counts and boundaries use `yolo_hybrid` (default) or `gemini`. If your observability report shows a `synthetic_motion_metadata` warning, this is why.

Q11: I re-ran a video and my previous results changed/disappeared. Why?

Videos are keyed by the **MD5 of the file's bytes**. Re-processing the *same* file is treated as a fresh ingest: `add_video` is `INSERT OR REPLACE`, and the foreign-key cascade **deletes the previous play/candidate segments** before writing the new ones. So the earlier results are intentionally replaced, not merged. (A `reingest_replaced_prior` note in the report flags this.) Note also that re-encoding or re-downloading a video changes its bytes ? a new MD5 ? a separate record.

Q12: What is the `*.report.json` / `*.report.md` / `*.summary.md` file next to my video?

Every processing run now writes a **per-video observability report** next to the video (or to `report.output_dir` if set in `config.json`):

- **`<name>.report.json`** ? machine-readable: every stage, metric, and finding. The source of truth; diff two runs to see what changed.
- **`<name>.report.md`** ? the **technical** report: validation results, processing stages, and **findings/warnings**. Each warning names the problem and cites the FAQ entry to read. **Start here when a run looks wrong.**
- **`<name>.summary.md`** ? a **customer-friendly** summary: just the video metadata and highlights, no system internals.

To read the technical report: scan **Verdict** first (PASS / PASS-with-warnings / FAIL), then **Findings & warnings** ? each finding has a plain-English message, evidence, and a `? see README#Qn` pointer. To turn reports off or relocate them, set `"report": {"enabled": false}` or `"report": {"output_dir": "..."}` in `config.json`.

Q13: A validation check failed and indexing was halted ? what do I do?

Open the report's **Processing stages ? validation** block (or the `VALIDATION SUMMARY` printed to the console). Each failed check states the exact reason. Common cases: **relevance** (the court color didn't match ? see [Q2](#q2-why-does-my-video-fail-the-relevance-check)); **resolution/fps** (below the configured minimums ? adjust `validation.min_*` in `config.json`); **format** (not an MP4 container); **blur** (footage too soft ? improve sharpness/lighting or lower `validation.min_blur_threshold`; a reading of `0.0` often means undecodable frames, so re-encode with `ffmpeg -i in.mp4 -c:v libx264 out.mp4` before lowering the threshold); **keyframes/PTS** (HEVC/GoPro keyframe quirks ? see [Q5](#q5-why-did-the-video-fail-with-no-keyframes--packet-timestamps-could-be-extracted)). Fix the underlying cause or relax the relevant threshold in `config.json`, then re-run.

Q14: The report says fps was a "fallback" (not probed) ? what does that mean?

The `Video` section shows `FPS | 30 (fallback)` and a `fps_fallback_used` warning when the pipeline could **not probe** the real frame rate from the file and fell back to a hardcoded default (30 fps). Because rally detection converts frames?seconds and scales motion thresholds by fps, a wrong fps **miscalibrates timing and motion sensitivity** ? segment boundaries and which motion counts as "play" can be off.

This usually means validation didn't run (it's the step that probes the file via `ffprobe`), or `ffprobe` couldn't read the stream's `r_frame_rate`. To fix: confirm the real fps ?

```bash
ffprobe -v error -select_streams v:0 -show_entries stream=r_frame_rate -of csv=p=0 "C:/path/to/video.mp4"
```

? make sure `ffprobe` is on your PATH (see [Q1](#q1-the-server-complains-that-ffprobe-is-not-found)) and that the run validates the file, then re-run so the real fps is used. (When fps shows `(probed)` instead, this is all fine.)

? Annotation & Evaluation

Once a video is processed, you can measure **how well the pipeline detected its rallies** against hand-annotated ground truth. You annotate each rally's `[start, end)` time range in a small CSV; the harness scores the detected rallies straight from the SQLite DB ? no API calls, fully deterministic.

```bash
python run_eval.py --scaffold --annotations rallies.csv          # make an empty template
# ...fill in start,end rows, then:
python run_eval.py --video "C:/videos/match.mp4" --annotations rallies.csv --show-failures
```

It reports precision/recall/F1, mean IoU, and boundary error for **two stages** ? raw detector `candidates` vs. AI-confirmed `final` segments ? which pinpoints whether a weak result is a *detection* problem or a *segmentation* problem.

> Rally timestamps evaluate and tune **segmentation**; they do **not** train the shuttle detector (that needs per-frame coordinate labels). See **docs/EVALUATION.md** for the annotation format, CLI flags, and how to read the output.

? Running the Test Suite

The test suite uses `pytest` and lives in `tests/`. From the project root:

```bash
pip install pytest pytest-cov --user

# One-liner full suite (recommended before opening a PR) ? discoverable wrapper that
# runs the whole suite, reports a clear PASS/FAIL, and returns pytest's exit code:
python run_all_tests.py
python run_all_tests.py -k yolo -v        # any args pass straight through to pytest

# ...or call pytest directly:
pytest                                # run everything
pytest tests/test_yolo_hybrid.py    # run a single file
pytest -k yolo --maxfail=1 -v       # filter by keyword, fail fast, verbose
pytest --cov=backend --cov-report=term-missing   # coverage report (uses .coveragerc)
```

```
362     > The optional `obsreport` package powers the per-video observability reports; its tests
363     > skip cleanly if it isn't installed. To exercise them: `pip install -e ../sports-
obsreport --no-build-isolation`.
364
365     The suite mocks external dependencies (the Gemini client, ffprobe/ffmpeg, OpenCV
`VideoCapture`, the YOLO model) so it runs offline and without a GPU. The tests cover:
366
367     *   `tests/test_base.py` ? segmenter registry
368     *   `tests/test_database.py` ? SQLite schema, video/segment insert/query
369     *   `tests/test_geometry.py` ? bbox center / distance / velocity / IoU helpers
370     *   `tests/test_video.py` ? ffprobe duration + ffmpeg chunk extraction helpers
371     *   `tests/test_providers.py` + `tests/test_provider_gemini.py` ? AI provider registry
and the Gemini provider
372     *   `tests/test_sport_profiles.py` ? sport profile registry + the badminton prompt
373     *   `tests/test_gemini.py` ? `GeminiPlaySegmenter` (single-file and chunked paths)
374     *   `tests/test_yolo_hybrid.py` ? `HybridYoloSegmenter` happy path, missing-API-key
bailout, and timestamp-validation drops
375     *   `tests/test_tracknet_runner.py` + `tests/test_tracknet_hybrid.py` ? TrackNet WSL
runner (mocked subprocess) and the `tracknet_hybrid` segmenter
376     *   `tests/test_eval_metrics.py` + `tests/test_eval_annotations.py` +
`tests/test_eval_harness.py` ? rally evaluation harness (interval matching, CSV loading, DB
scoring, CLI)
377
378     ---
379
380     ## ? Project Structure (high level)
381
382     ```
383     backend/
384       main.py                # FastAPI app + CLI orchestrator
385       results.py             # Run-result aggregation
386       pipeline/              # The processing pipeline
387         segmenters/          # Pluggable rally segmenters: motion, gemini, yolo_hybrid,
tracknet_hybrid, wasb_hybrid, fusion_hybrid
388         detectors/           # Shuttle/person detectors: tracknet, wasb, person
389         validators/          # Pluggable video validators (format, blur, scene-cut,
relevance, ...)
390         stitcher/compiler.py  # FFmpeg-based highlight compiler
391       api/static/            # Zero-build frontend served by FastAPI
392       infrastructure/        # SQLite layer (database.py) + execution policy
393       providers/             # Pluggable AI providers (gemini today)
394       sports/                # Pluggable sport profiles (badminton today)
395       eval/                  # Rally evaluation harness: metrics, annotations, scoring
396       annotations/           # Rally annotation providers
397       config/                # Config schema + model registry
398       features/              # Rally sequence features
399       reporting/             # Observability run signals
400       tools/                 # Export + rally slicing utilities
401       utils/                 # ffprobe/ffmpeg + bbox geometry helpers
402     docs/                    # Live docs (CODE_MAP, COMMERCIALIZATION, BACKLOG, ROADMAP,
guidelines, golden-set). Historical items in docs/archives/
403     tests/                   # pytest suite (mocked, runs offline)
404     scratch/                 # Throwaway experiments
405     training/                # Offline detector training (see ROADMAP)
406     eval_baselines/          # Frozen evaluation baselines
407     scripts/
408       doctor.py              # Dependency & FFmpeg diagnostics + default config init
409       run_process_and_stitch.py # Interactive end-to-end runner
410     tools/
411       cleanup_caches.py      # Dry-run-first cache GC (WSL ~/clips + output/); see "Cache
Hygiene" above
412     run_all_tests.py         # Full test-suite wrapper (see "Running the Test Suite" above)
413     run_eval.py              # Rally evaluation CLI (see "Annotation & Evaluation" above)
414     run_regression.py        # Regression runner
415     pyproject.toml           # PEP 621 packaging (hatchling); `indexer` entry point + deps
```

```
416     config.json              # Runtime configuration (see "Threshold Tuning" above)
417     ```
418
```

11. user (2026-06-15T14:58:05.420Z)

```
README.md:28:*   **High-Speed FFmpeg Stitching** with configurable `begin_padding`,
`end_padding`, and `min_shot_count` filtering, plus an optional **branding watermark** burned
into every reel (**on by default** ? `"Generated by Khelsutra.guru"`, fully configurable, with
an off switch).
README.md:119:### 4. Extracting a Trimmed Segment (`--trim-start` / `--trim-end`) ? also the
quickest watermark preview
README.md:120:Extract a slice of a recording to disk (re-encoded; and, with the default-on
branding watermark, stamped **"Generated by Khelsutra.guru"** in the corner):
README.md:125:Output lands in `--output-dir` (default `output/`) unless `--trim-output` is an
absolute path. This is the fastest way to *see* the watermark on a real clip ? set
`stitching.watermark.enabled: false` in `config.json` to turn it off.
README.md:129:> ffmpeg -i in.mp4 -vf "drawtext=text='Generated by Khelsutra.guru':font='DejaVu S
ans':fontcolor=white@0.85:fontsize=h/28:x=w-tw-24:y=h-th-
24:box=1:boxcolor=black@0.4:boxborderw=10" -c:a copy out.mp4
README.md:195:        "watermark": {
README.md:197:            "text": "Generated by Khelsutra.guru",
README.md:218:[Omitted long matching line]
scripts\run_process_and_stitch.py:144:
watermark=cfg.stitching.watermark)
config.json:77:        "watermark": {
config.json:79:            "text": "Generated by Khelsutra.guru",
backend\assets\fonts\README.md:3:Fonts shipped with the highlight **watermark**
(`stitching.watermark`). All bundled fonts
backend\assets\fonts\README.md:11:`RobotoSlab-Regular.ttf` is the **default** watermark font:
`stitching.watermark.font_path`
backend\assets\fonts\README.md:14:bundled one ? set `stitching.watermark.font_path` to its path
in `config.json`.
backend\assets\fonts\LICENSE-Apache-2.0.txt:115:          within a display generated by the
Derivative Works, if and
backend\main.py:108:    On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so
only the
backend\main.py:109:    fresh segments remain. On an empty run, drop the NEW partial rows (id >
watermark) and
backend\main.py:189:        play_wm, cand_wm = db_instance.segment_watermarks(video_id)
backend\main.py:618:    # Run compiler with the configured stitching paddings + optional
branding watermark
backend\main.py:621:                                end_padding=stitching.end_padding,
watermark=stitching.watermark)
backend\main.py:752:        # clip carries the same configured stitching paddings + watermark as
a real
backend\main.py:753:        # highlight (watermark default-on).
backend\main.py:756:                                    end_padding=stitching.end_padding,
watermark=stitching.watermark)
backend\main.py:833:            play_wm, cand_wm = db_instance.segment_watermarks(video_id)
backend\pipeline\stitcher\compiler.py:10:# Bundled, commercially-clean default watermark font
(Apache-2.0). Shipped so the overlay
backend\pipeline\stitcher\compiler.py:12:# `font='Sans'` fails). Overridable via
stitching.watermark.font_path. This module lives at
backend\pipeline\stitcher\compiler.py:18:                    watermark: Optional[Any] = None):
backend\pipeline\stitcher\compiler.py:22:        # Optional branding overlay. Accepts a
WatermarkConfig model or a plain dict;
backend\pipeline\stitcher\compiler.py:24:        self.watermark =
self._normalize_watermark(watermark)
backend\pipeline\stitcher\compiler.py:28:    def _normalize_watermark(watermark: Optional[Any])
-> Optional[Dict[str, Any]]:
backend\pipeline\stitcher\compiler.py:29:        if watermark is None:
backend\pipeline\stitcher\compiler.py:31:        if hasattr(watermark, "model_dump"):
backend\pipeline\stitcher\compiler.py:32:            return watermark.model_dump()
backend\pipeline\stitcher\compiler.py:33:        if isinstance(watermark, dict):
```

```
backend\pipeline\stitcher\compiler.py:34:            return dict(watermark)
backend\pipeline\stitcher\compiler.py:47:        on-by-default bundled-font watermark (and
absolute logo paths) on Windows."""
backend\pipeline\stitcher\compiler.py:98:    def _watermark_filter(self, tmp_dir: str) ->
Optional[str]:
backend\pipeline\stitcher\compiler.py:99:        """Build the ffmpeg -vf filtergraph that burns
the configured watermark (text
backend\pipeline\stitcher\compiler.py:101:        watermarking is disabled or has nothing to
draw. The concat stage stays -c copy
backend\pipeline\stitcher\compiler.py:103:        wm = self.watermark
backend\pipeline\stitcher\compiler.py:110:            logger.warning(f"[watermark] logo not
found: {logo_path} ? skipping logo overlay.")
backend\pipeline\stitcher\compiler.py:137:            tf_path = os.path.join(tmp_dir,
"watermark.txt")
backend\pipeline\stitcher\compiler.py:192:            # Build the (optional) watermark
filtergraph once; it is applied during every
backend\pipeline\stitcher\compiler.py:194:            watermark_filter =
self._watermark_filter(tmp_dir)
backend\pipeline\stitcher\compiler.py:195:            if watermark_filter:
backend\pipeline\stitcher\compiler.py:196:                logger.info("[watermark] applying
branding overlay to each clip.")
backend\pipeline\stitcher\compiler.py:254:                vf_args = ["-vf", watermark_filter] if
watermark_filter else []
backend\pipeline\stitcher\compiler.py:256:                # Try with the watermark first; if
that encode fails (e.g. a bad font/logo
backend\pipeline\stitcher\compiler.py:273:                                    f"{segment_label}:
watermark encode failed; retrying without it. "
backend\pipeline\stitcher\compiler.py:281:
logger.warning(f"{segment_label}: encoded WITHOUT the watermark (the watermark filter failed).")
docs\UI_PROPOSAL.md:1:# UI Proposal ? khelsutra.guru web app (alpha ? beta)
docs\UI_PROPOSAL.md:34:API = existing FastAPI on the GPU box via tunnel at `api.khelsutra.guru`.
docs\UI_PROPOSAL.md:98:2. **RESOLVED 2026-06-10:** product name = **KhelSutra Guru**; domain =
**khelsutra.guru** (GoDaddy receipt, 3-yr registration + domain protection, renews 2029-06).
docs\UI_ALPHA_EXECUTION_PLAN.md:12:| 2026-06-10 | Product name **KhelSutra Guru** · domain
**khelsutra.guru** (GoDaddy, 3-yr + protection) |
docs\UI_ALPHA_EXECUTION_PLAN.md:48:- [ ] **D1 (owner):** Cloudflare account ? add khelsutra.guru
? GoDaddy nameserver switch (HOSTING_PLAN §5) ? scoped API token (§6) ? Pages project connected
to GitHub `khelsutra` (build root `web/`)
docs\UI_ALPHA_EXECUTION_PLAN.md:49:- [ ] **D2 (Claude):** DNS records; `cloudflared` Windows
service on the box ? `api.khelsutra.guru`; Access apps + email-allowlist policies for `app` &
`api`; SPA prod env
backend\infrastructure\database.py:256:    def segment_watermarks(self, video_id: str) ->
Tuple[int, int]:
backend\infrastructure\database.py:259:        Captured before a (re)segmentation so the run's
new rows (id > watermark) can be
backend\infrastructure\database.py:260:        told apart from prior rows (id <= watermark),
enabling destroy-AFTER-confirm.
backend\infrastructure\database.py:276:        successful run (``*_upto`` = the pre-run
watermark), or drop the NEW partial rows
tests\test_api_endpoints.py:207:        def __init__(self, out_dir, watermark=None, **kwargs):
tests\test_api_endpoints.py:272:    cfg.stitching.watermark.enabled = False  # keep the trim
command minimal
backend\config\models.py:172:class WatermarkConfig(BaseModel):
backend\config\models.py:175:    (under `stitching.watermark` in config.json) to turn it off.
The text is fully
backend\config\models.py:178:    text: str = "Generated by Khelsutra.guru"
backend\config\models.py:194:    watermark: WatermarkConfig =
Field(default_factory=WatermarkConfig)
docs\STORAGE_SHARING_MODEL.md:62:A Gmail-style **quota meter** is permanent UI. Profile =
`khelsutra.guru/@handle`: reel gallery,
docs\STORAGE_SHARING_MODEL.md:184:12. Profile namespace: `@handles` on khelsutra.guru?
docs\EXECUTION_POLICY.md:102:  watermark (or skip the prune on park) so a resume does not redo
confirmed work.
docs\CODE_MAP.md:7:[Omitted long matching line]
docs\CODE_MAP.md:186:[Omitted long matching line]
```

docs\CODE_MAP.md:211:[Omitted long matching line]
docs\CODE_MAP.md:291:- **`add_video` is an UPSERT** (`INSERT ... ON CONFLICT(id) DO UPDATE`)
that mutates the row in place ? re-ingesting a `video_id` does NOT cascade-delete its segments
(the old `INSERT OR REPLACE` + FK-cascade wipe is gone). Segment clearing is now an explicit
destroy-AFTER-confirm via `segment_watermarks`/`prune_segments` (or `clear_video_data`).
`_get_connection` MUST still re-issue `PRAGMA foreign_keys=ON` per connection (CASCADE/SET NULL
still apply on an explicit delete).
docs\KHELSUTRA_PRODUCT_OVERVIEW.md:45: *"Generated by KhelSutra.guru"* mark.
docs\KHELSUTRA_PRODUCT_OVERVIEW.md:165:*KhelSutra · khelsutra.guru · Internal & invited-trial
overview, June 2026.*
docs\archives\decisions\DECISIONS.md:477:1. **Product name: KhelSutra Guru**; domain
khelsutra.guru (GoDaddy receipt: 3-yr
docs\NEXT_STEPS.md:5:ADR-009 ratified ? KhelSutra Guru / khelsutra.guru, private `khelsutra`
repo, local-first delivery ? see
docs\I18N_PLAN.md:111:The bundled UI/watermark fonts (Outfit, Roboto Slab) **do not cover
Kannada or Devanagari**.
docs\I18N_PLAN.md:113:with the watermark font: OFL is not in the strict **MIT/Apache/BSD**
code/data/weights guardrail.
docs\I18N_PLAN.md:118:Note the **video watermark already takes a `font_path`**
(`WatermarkConfig.font_path`,
docs\I18N_PLAN.md:119:`backend/pipeline/stitcher/compiler.py`), so stamping a Kannada/Hindi
watermark is a
docs\I18N_PLAN.md:180:   also unblocks the **Kannada/Hindi video watermark** (needs a Noto
`font_path`).
docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:73:- Both paths duplicate:
hashing, validation, add_video, segmenter lookup, watermark + destroy-after-confirm,
`_finalize_reingest`, `emit_indexer_report` in finally, etc.
docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:77:- 2026-06-10 hardening
(detector keying by video_id/MD5, re-ingest destroy-AFTER-confirm via watermarks+prune,
WAL+busy_timeout, typed wasb config, ExecutionPolicy, path safety helpers, single
`compute_video_id`, trajectory twin dedup into `TrajectoryHybridSegmenter`, person cue
extraction to reusable producer) is visibly landed and tested.
docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:185:7. **Re-ingest safety**:
Re-submit the same video (different segmenter or forced failure). Confirm prior good
segments/candidates are preserved on failure/empty and only replaced after successful
confirmation (watermark + prune contract).
docs\HOSTING_PLAN.md:1:# Hosting Plan ? khelsutra.guru
docs\HOSTING_PLAN.md:27:   ??? app.khelsutra.guru ? Cloudflare Pages (React SPA; auto-deploy
from GitHub)
docs\HOSTING_PLAN.md:28:   ??? api.khelsutra.guru ? Cloudflare Tunnel (outbound-only
`cloudflared` service on the box)
docs\HOSTING_PLAN.md:49:browser ??? Cloudflare edge (free, unchanged) ??? app.khelsutra.guru
(Pages)
docs\HOSTING_PLAN.md:50:   ??? api.khelsutra.guru ???
Cloud Run (FastAPI control plane, scale-to-zero)
docs\HOSTING_PLAN.md:80:1. Create a free Cloudflare account ? **Add a domain** ?
`khelsutra.guru` ? Free plan. Cloudflare
docs\HOSTING_PLAN.md:82:2. GoDaddy ? **My Products ? khelsutra.guru ? Manage DNS ? Nameservers ?
Change ?
docs\HOSTING_PLAN.md:104:Access allowlist ON for both hostnames ? CORS narrowed to
`https://app.khelsutra.guru` ? rate
tests\test_reingest_safety.py:32:   play_wm, cand_wm = db.segment_watermarks("vid1")
tests\test_reingest_safety.py:47:   play_wm, cand_wm = db.segment_watermarks("vid1")
tests\test_reingest_safety.py:58:def test_watermarks_zero_when_empty(tmp_path):
tests\test_reingest_safety.py:61:   assert db.segment_watermarks("vid1") == (0, 0)
tests\test_reingest_safety.py:75:   play_wm, cand_wm = db.segment_watermarks("vid1")
tests\test_watermark.py:1:"""Tests for the configurable highlight watermark
(stitching.watermark).
tests\test_watermark.py:5:ffmpeg/ffprobe mocked ? including the graceful fallback when the
watermark encode fails.
tests\test_watermark.py:16:from backend.config.models import WatermarkConfig
tests\test_watermark.py:22:def test_watermark_config_defaults_on_with_brand_text():
tests\test_watermark.py:23:   wm = AppConfig().stitching.watermark
tests\test_watermark.py:25:   assert wm.text == "Generated by Khelsutra.guru"
tests\test_watermark.py:29:def test_watermark_can_be_disabled_via_config():

```
tests\test_watermark.py:30:    cfg = AppConfig.model_validate({"stitching": {"watermark":
{"enabled": False}}})
tests\test_watermark.py:31:    assert cfg.stitching.watermark.enabled is False
tests\test_watermark.py:36:        WatermarkConfig(opacity=1.5)
tests\test_watermark.py:38:        WatermarkConfig(font_size_ratio=0.0)
tests\test_watermark.py:44:    assert VideoCompiler(str(tmp_path / "a"),
watermark=None)._watermark_filter(str(tmp_path)) is None
tests\test_watermark.py:45:    off = VideoCompiler(str(tmp_path / "b"), watermark={"enabled":
False})
tests\test_watermark.py:46:    assert off._watermark_filter(str(tmp_path)) is None
tests\test_watermark.py:50:    custom = "My Academy 2026 ? Khelsutra.guru"
tests\test_watermark.py:51:    comp = VideoCompiler(str(tmp_path / "o"), watermark={"enabled":
True, "text": custom})
tests\test_watermark.py:52:    graph = comp._watermark_filter(str(tmp_path))
tests\test_watermark.py:55:    assert (tmp_path / "watermark.txt").read_text(encoding="utf-8")
== custom
tests\test_watermark.py:65:                            watermark={"enabled": True, "text": "X",
"position": "top-left", "margin": 12})
tests\test_watermark.py:66:    graph = comp._watermark_filter(str(tmp_path))
tests\test_watermark.py:72:                            watermark={"enabled": True, "text": "X",
"font_path": "/fonts/Brand.ttf"})
tests\test_watermark.py:73:    g = comp._watermark_filter(str(tmp_path))
tests\test_watermark.py:79:    comp = VideoCompiler(str(tmp_path / "o2"), watermark={"enabled":
True, "text": "X"})
tests\test_watermark.py:80:    g = comp._watermark_filter(str(tmp_path))
tests\test_watermark.py:87:    assert cmod._BUNDLED_FONT.exists(), "bundled default watermark
font must ship in the repo"
tests\test_watermark.py:98:                            watermark={"enabled": True, "text": "X",
"font": "DejaVu Sans"})
tests\test_watermark.py:99:    g = comp._watermark_filter(str(tmp_path))
tests\test_watermark.py:107:# bundled-font watermark (and absolute logo paths) on Windows.
Verified end-to-end
tests\test_watermark.py:118:def test_watermark_font_path_with_drive_colon_is_escaped(tmp_path):
tests\test_watermark.py:121:                            watermark={"enabled": True, "text": "X",
"font_path": "C:/fonts/Brand.ttf"})
tests\test_watermark.py:122:    g = comp._watermark_filter(str(tmp_path))
tests\test_watermark.py:131:                            watermark={"enabled": True, "text": "X",
"logo_path": str(logo)})
tests\test_watermark.py:132:    graph = comp._watermark_filter(str(tmp_path))
tests\test_watermark.py:140:                            watermark={"enabled": True, "text": "X",
"logo_path": str(tmp_path / "nope.png")})
tests\test_watermark.py:141:    graph = comp._watermark_filter(str(tmp_path))
tests\test_watermark.py:161:def
test_compile_applies_watermark_vf_to_each_clip_not_concat(tmp_path):
tests\test_watermark.py:165:                            watermark={"enabled": True, "text":
"Generated by Khelsutra.guru"})
tests\test_watermark.py:180:def test_compile_without_watermark_adds_no_vf(tmp_path):
tests\test_watermark.py:183:    comp = VideoCompiler(str(tmp_path / "out"))  # no watermark
passed
tests\test_watermark.py:190:def
test_watermark_failure_falls_back_to_unwatermarked_clip(tmp_path):
tests\test_watermark.py:191:    """A bad font/logo must not nuke the reel: the clip re-encodes
without the watermark."""
tests\test_watermark.py:195:                            watermark={"enabled": True, "text":
"Generated by Khelsutra.guru"})
tests\test_watermark.py:200:    # Reel still produced despite the watermark encode failing.
tests\test_watermark.py:203:    # For the clip, both a watermarked attempt and a plain fallback
attempt were made.
tests\test_watermark.py:205:    assert any("-vf" in c for c in clip_cmds)       # watermark
attempt
tests\test_watermark.py:209:def test_watermark_failure_logs_the_ffmpeg_reason(tmp_path, caplog):
tests\test_watermark.py:215:    comp = VideoCompiler(str(tmp_path / "out"),
watermark={"enabled": True, "text": "Y"})
```

12. assistant (2026-06-15T14:58:13.253Z)

I have enough cross-referenced ground truth. Let me note one important finding: the watermark text. Let me verify the exact casing discrepancy.

The draft says `"Generated by KhelSutra.guru"` (line 45) ? capital S in "KhelSutra". The actual shipped watermark text (README line 28/197, config.json line 79, `backend/config/models.py:178`, tests) is `"Generated by Khelsutra.guru"` ? lowercase "s" in "Khelsutra". This is a factual mismatch with what the product actually stamps on reels.

Let me also verify the re-ingest behavior claim since the draft says "process once, come back to it" and there's a known UPSERT vs INSERT-OR-REPLACE nuance, and check whether the doc's "AI assistant coach" / flywheel framing matches the owned-model roadmap (which does NOT auto-fine-tune per user). I have what I need from NEXT_STEPS, COMMERCIALIZATION, and the CLAUDE.md guardrails. Let me do a final check on the multi-court "0.66" framing and whether the doc leaks technical detail.

I have all the ground truth needed. Compiling the structured review.

13. user (2026-06-15T14:58:58.098Z)

Output does not match required schema: root: must have required property 'verdict', root: must have required property 'must_fix', root: must have required property 'should_fix', root: must have required property 'nice_to_have', root: must have required property 'praise'

14. user (2026-06-15T14:59:47.182Z)

Structured output provided successfully

15. assistant (2026-06-15T14:59:53.688Z)

I've completed the grounding/factual-accuracy review of `C:/Users/avidu/Projects/badminton-highlight-indexer/docs/KHELSUTRA_PRODUCT_OVERVIEW.md` against the repo's ground truth (NEXT_STEPS.md, COMMERCIALIZATION.md, README.md, FIELD_VISIT_PLAYBOOK.md, PRODUCT_FIELD_ASSOCIATE_FLYER.md, config.json, backend/config/models.py, and CLAUDE.md guardrails).

Two must-fix factual issues: the watermark text is mis-cased ("KhelSutra.guru" in the draft vs the actually-shipped "Khelsutra.guru"), and the flywheel section overstates the mechanism as a live, per-user self-improving model when the roadmap is a future, opt-in, batch process on a shared consented corpus with no per-user fine-tuning. The rest of the doc ? today-vs-roadmap split, capture rules, multi-court framing, and the opt-in/minors/local-first guardrails ? is accurate and honest.